

Claude Code实操-企业多模态知识库

本课程围绕 Claude Code 实践企业级多模态知识库系统，讲解从需求分析到落地的全流程。课程以 GraphRAG 为核心，结合向量检索与知识图谱，基于 MinerU、LangExtract、LangChain 搭建 Agentic-RAG 体系，实现多源文档解析与智能问答。

整个开发过程**严格遵循 SDD 规范驱动开发**，先定接口与组件规范，再通过 MVP 多阶段验证，保障模块解耦与稳定对接；后续按规范完成前后端分离、接口设计与联调，最终实现可视化交互与完整可用的知识库系统，充分体现了**SDD 在工程化落地、质量可控与高效协作中的关键作用**。

1、项目需求

在本章中，我们将从零开始，借助 Claude Code 工具，完整开发一套企业级多模态知识库系统。系统的整体实现效果可参考 ChatPDF 等平台：支持 PDF、DOCX、CSV 等多种格式文件的上传与解析，自动完成文件内容的分片、提取与索引构建，并提供直观的前端交互界面，让用户能够针对已上传的文档进行精准问答。



接下来，我将手把手地带大家借助 Claude Code 完成这一企业级项目的开发。在开发过程中，大家需要重点关注 Vibe Coding 的流程，这与传统开发方式略有不同。

具体开发流程如下：

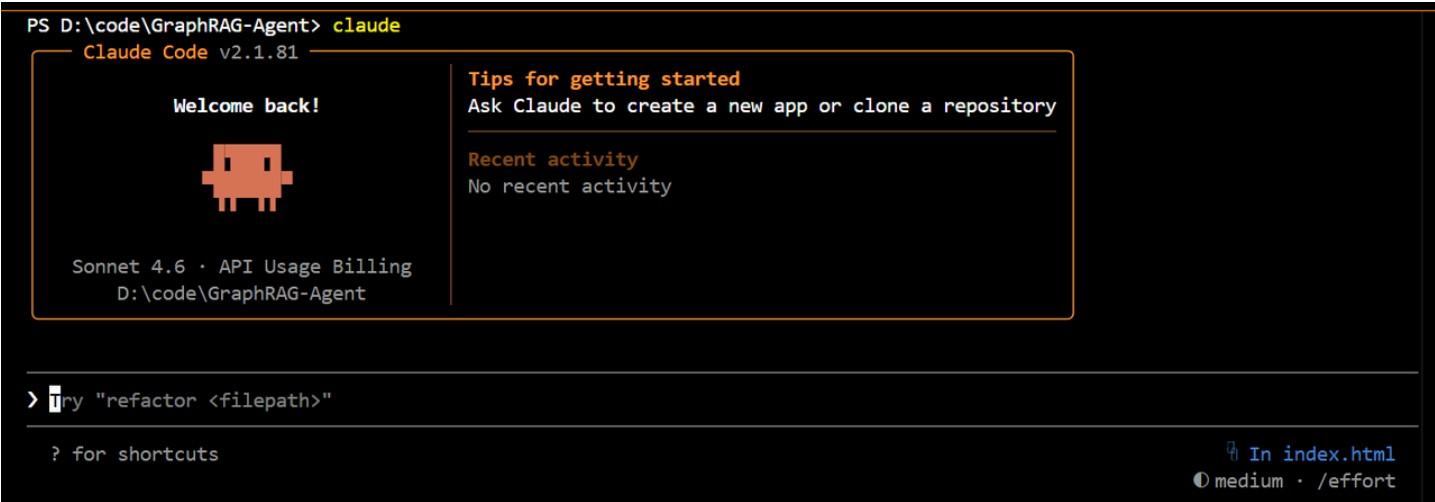
1. 需求分析梳理
2. 借助 AI 工具进行架构设计
3. 借助 AI 工具进行技术选型

- 4. 借助 AI 工具进行技术调研
- 5. 进行 MVP 测试（关键环节，涉及前后端接口规范的设计）
- 6. 基于 MVP 测试生成的接口规范文档，结合需求产出 PRD 文档
- 7. 基于 PRD 文档生成前端代码
- 8. 基于 PRD 文档生成后端代码
- 9. 前后端联调

- 学习目标：
- 1) 学会熟练使用 Claude Code
 - 2) 掌握企业多模态知识库的开发思路
 - 3) 驾驭工程在 Vibe Coding 中的落地（SDD 规范驱动编程）

2、项目准备

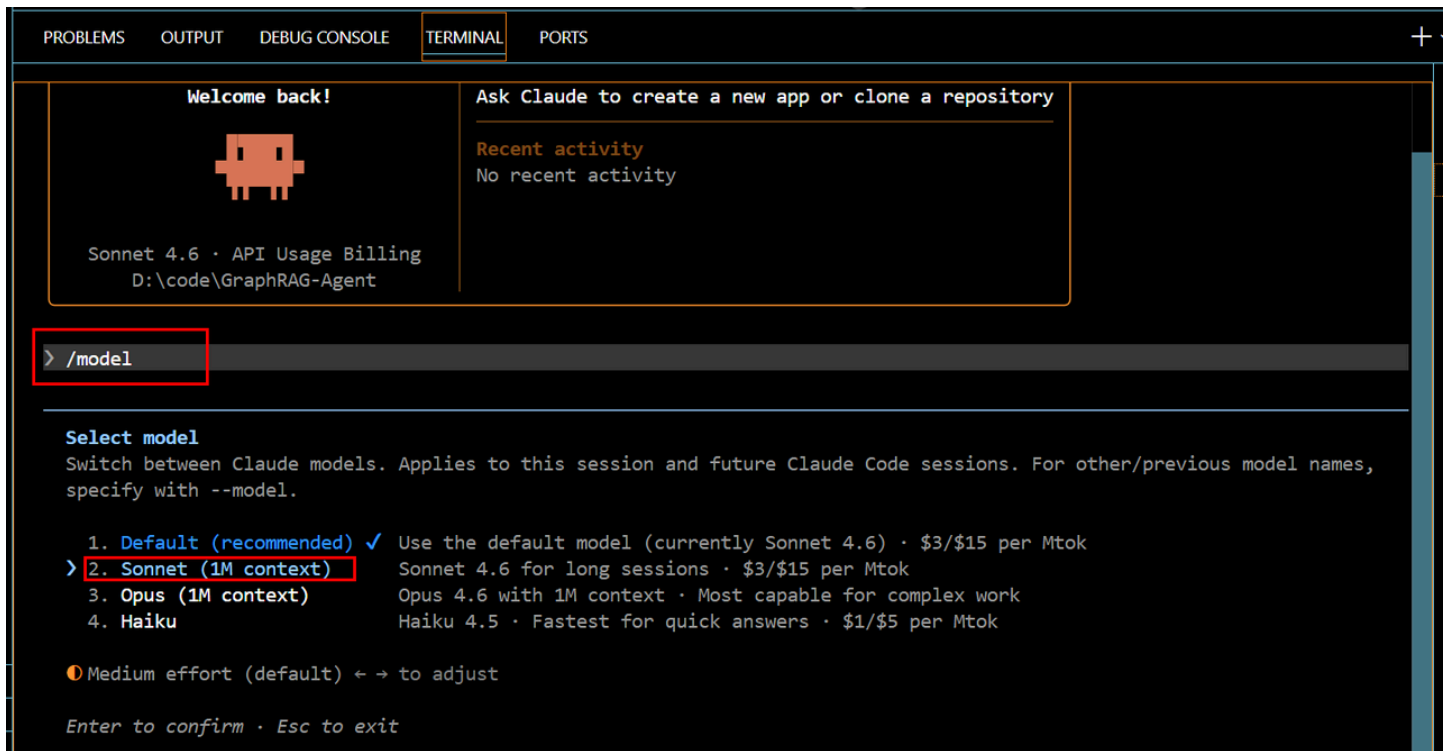
首先在电脑上**选择一个合适的位置**，创建本次开发的**项目根目录**，项目名称固定为：**GraphRAG-Agent**。我当前使用的是 **Windows 系统** 搭建 Claude Code 开发环境，**Mac、Linux 等其他操作系统的操作步骤基本一致**，大家可以参照执行。目录创建完成后，**进入这个新建的项目目录**，后续所有开发操作都将在该目录下进行。



为了能让我们顺利 Vibe Coding 首先给大家分享几个 Claude Code 工具的高频技巧。

2.1 Claude Code 技巧1

在正式开始开发之前，我们先通过 Claude Code 内置的**斜杠命令**确认本次开发使用的 AI 大模型。你只需要输入 `/model` 命令，就能查看并切换可用模型。我这里选择的是综合能力更强的 **Sonnet 模型**，大家可以根据自己的账号权限和需求，选择合适的模型即可。



2.2 Claude Code 技巧2

在我们的开发过程中，Claude Code 提供了**三种权限控制模式**，方便你灵活调整它的操作权限。你可以直接按下快捷键 **Shift + Tab**，在三种模式之间**循环切换**，**当前选中的模式**会在界面上高亮显示，一眼就能看清当前状态。

1、默认模式（Default Mode）

- **核心行为**：在执行**文件编辑**和**系统命令**前，都会向你弹出确认窗口，需要你手动点击“同意”后才会执行。
- **适用场景**：
 - 刚接触 Claude Code，希望严格把控每一步操作
 - 处理敏感代码或生产环境相关操作
 - 不确定 AI 生成的代码是否安全，需要人工审核
- **特点**：最安全、最保守的模式，几乎不会出现误操作，但会打断开发流程，效率稍低。

2、自动接受编辑（Auto-accept Edits）

- **核心行为**：
 - **文件编辑**：AI 可以直接修改、创建、删除代码文件，无需你确认
 - **系统命令**：执行终端命令（如安装依赖、运行脚本）前仍会弹窗询问，需要你手动确认
- **适用场景**：
 - 日常开发中，AI 辅助重构、补全代码
 - 你信任 AI 生成的代码变更，希望减少确认弹窗、提升编码速度
 - 只对终端命令这类高风险操作保持审核

- **特点：**兼顾效率与安全，是很多开发者的常用模式。

3、Plan Mode（计划模式）

- **核心行为：**
 - 仅使用**只读工具**（如查看文件、搜索代码、分析项目结构）
 - 不会直接修改文件或执行任何命令
 - 会生成一份详细的**可批准执行计划**，列出所有要做的变更步骤，由你手动审核后再决定是否执行
- **适用场景：**
 - 先让 AI 梳理方案、评估影响，再决定是否落地
 - 处理大型重构、复杂功能开发，需要先做可行性分析
 - 完全不想让 AI 触碰代码，只做“顾问”角色
- **特点：**最轻量化、无风险的模式，适合做前期调研和方案设计。



2.3 Claude Code 技巧3



这三项技巧是提高 **Claude Code**（Anthropic 推出的命令行 AI 编程助手）使用效率的核心。它不仅仅是一个聊天框，更像一个深度集成在你开发环境里的“结对编程”伙伴。

以下是这三个技巧的深度解析：

1. 中断与引导：夺回控制权

很多时候 AI 生成代码就像“开倒车”，一旦发现它理解偏了，千万别坐等它写完。

- **即时纠偏**：当你看到 Claude 开始生成错误的代码逻辑或不符合预期的文件时，你可以直接输入更正内容并按 **Enter**。
- **为什么高效**：在传统的网页版 AI 中，你通常需要点击“停止”按钮，然后重新输入。而在 Claude Code 中，这种“中断-注入新指令”的操作是无缝的。它会立即停下手中的活，根据你刚给的提示进行调整，**不需要重新开始对话**，极大地节省了 Token 和等待时间。

2. 支持图像：视觉化你的 Bug

“一张图胜过千言万语”在调试 UI 或理解复杂架构时尤其有效。

- **多样的输入方式**：
 - **拖拽/粘贴**：像操作聊天软件一样简单（Windows 用户注意是 **Alt + V**）。
 - **路径引用**：如果你在终端里，直接输入图片路径也是可以的。
- **典型应用场景**：
 - **UI 复刻**：直接给它一张设计稿截图，让它帮你写 CSS 或 React 组件。
 - **错误排查**：把浏览器报错的截图发给它，它能结合你的代码库分析出到底是哪一行出了问题。

- **图表理解**：发一张数据库架构图，让它根据图示编写 SQL 查询。

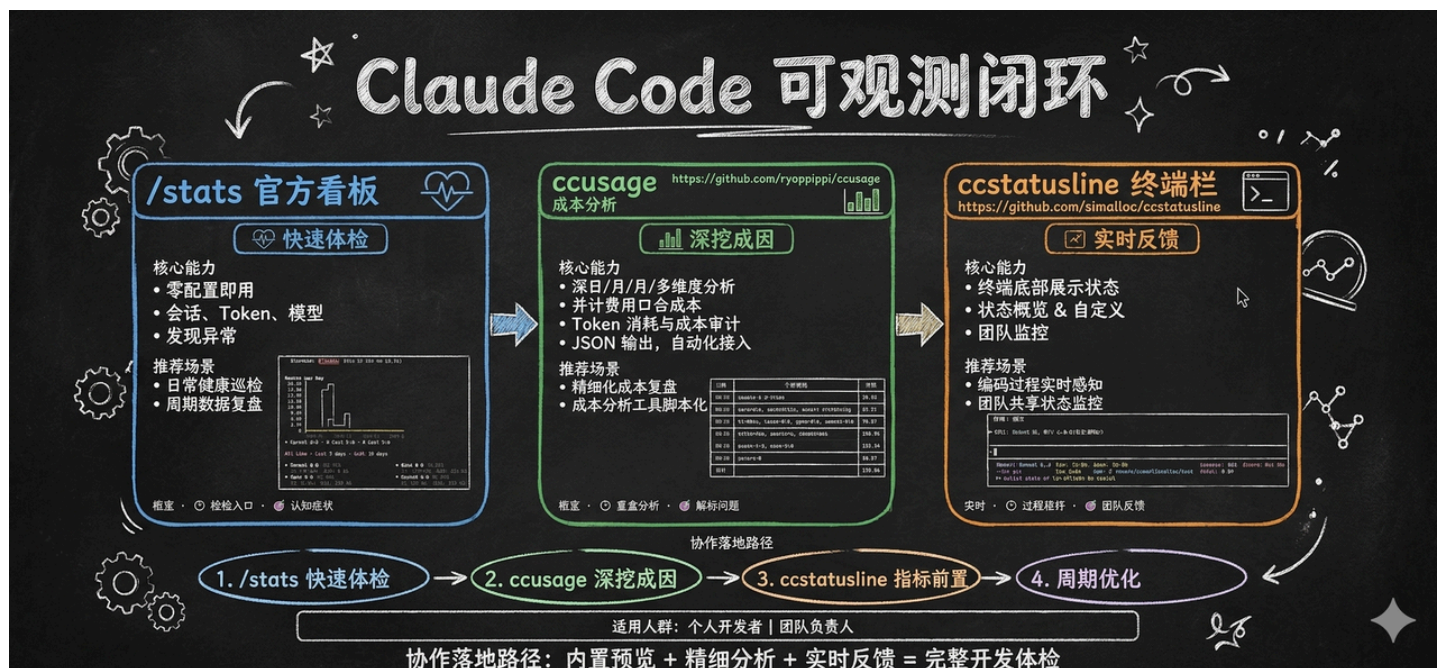
3. 使用 @ 引用：精准定位上下文

这是 Claude Code 的“杀手锏”功能，解决了 AI 面对庞大代码库时“找不着北”的问题。

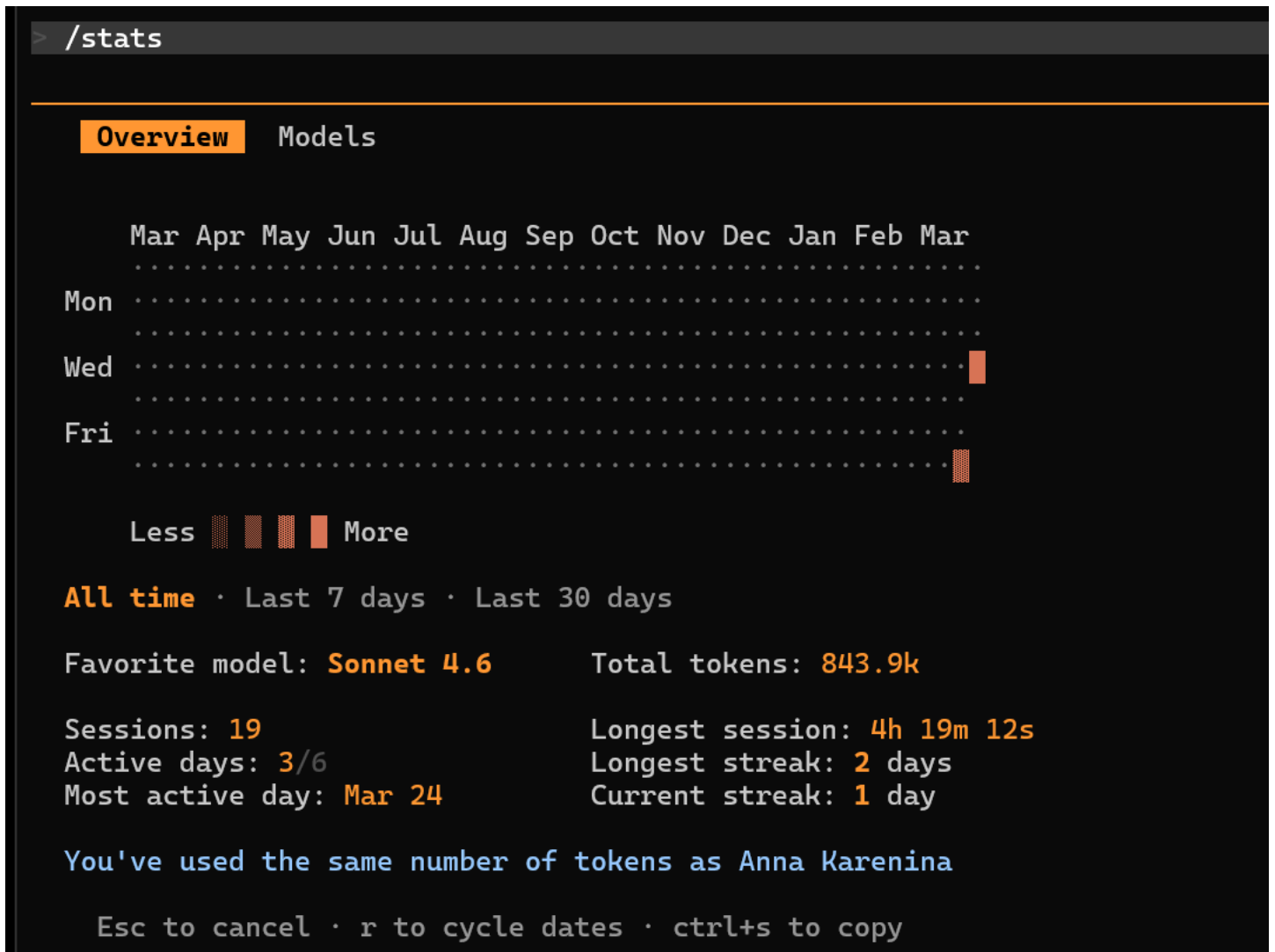
- **原理**：通过 @ 符号，你可以显式地告诉 Claude：“现在只看这几个文件”。
- **引用级别**：
 - **@文件**：例如 `@src/utils/auth.js`。Claude 会立即读取该文件的最新内容。
 - **@目录**：例如 `@src/components`。它会扫描该目录下的结构，帮助它理解组件间的调用关系。
- **核心优势**：它避免了“上下文污染”。如果你让 Claude 优化性能，但没告诉它具体的类，它可能会乱翻一通；用了 @，就像给它戴上了手术显微镜，注入速度极快且回答准确度直线攀升。

2.4 Claude Code 技巧4

如果你希望直观、可视化地查看 Claude Code 的运行状态与核心指标，可以使用下面这款工具：



Claude Code 自带了一个非常实用的斜杠命令工具，输入 `/stats` 就可以直接查看当前的 Token 使用统计信息。



如果你希望查看更清晰、更美观的可视化展示效果，可以安装这款开源工具：
<https://github.com/ryoppippi/ccusage> 它能让 Claude Code 的使用数据展示更加直观。

代码块

```
1  ## 在Claude Code里执行
2  https://github.com/ryoppippi/ccusage  给我在本地安装这个插件
```

效果如下：



如果想要在 Claude Code 的命令行界面下方，实时展示当前运行模式、各类核心运行指标等信息，让数据查看更直观便捷，大家可以安装这款开源工具：<https://github.com/sirmalloc/ccstatusline>

代码块

1

在Claude Code里执行

2

给我安装一下这个工具：<https://github.com/sirmalloc/ccstatusline>

代码块

1

在Claude Code里执行

2

帮我把默认配置写入 settings

效果如下：

```
写入默认配置到 settings
23 +   "type": "command",
24 +   "command": "ccstatusline"
25 + },
26   "model": "sonnet[1m]"
27 }
```

我再确认一下写入结果。

Read 1 file (ctrl+o to expand)

已经写好了，全局配置在：

C:\Users\ZhuanZ\.claude\settings.json:21

新增内容是：

```
"statusLine": {
  "type": "command",
  "command": "ccstatusline"
}
```

如果当前界面还没显示出来，重启一下 Claude Code，或者重新开一个会话即可生效。

Churned for 2m 22s

Model: Sonnet 4.6 (1M context) | Ctx: 0 | no git | (no git)

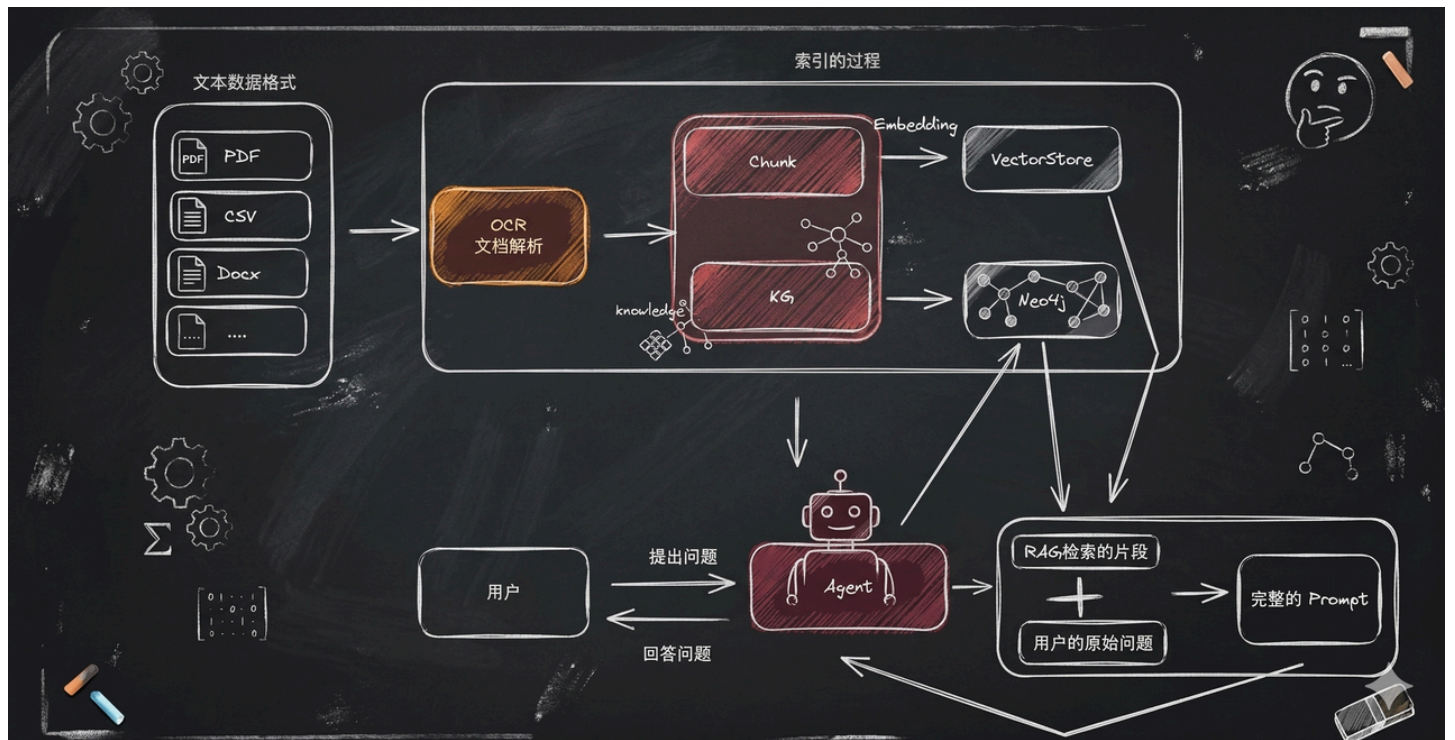
上面提到的这类辅助工具并非必需组件，即使不进行安装，也不会对我们本次项目的正常开发造成任何影响，大家可以根据自身需求灵活选择是否使用。

3、架构设计

接下来，我们就根据前面确定的项目需求，来梳理整体的项目架构。设计架构主要有两种实用思路，大家可以根据自身情况选择：

1. 如果你本身有比较丰富的开发经验，可以直接根据需求，自主设计整套项目架构。
2. 如果你在架构设计方面经验相对不足，可以把完整的项目需求交给 ChatGPT、Gemini、DeepSeek 这类大模型，让它们帮你深度分析并生成专业的技术方案，之后你再从中挑选合适的方案使用即可。

本次课程的核心是为大家演示 **Claude Code 这款工具** 的实际使用流程，因此我们就不再借助其他大模型生成方案，而是直接结合自身开发经验，手动完成项目架构的设计。



我们需要构建 **GraphRAG（图增强的检索增强生成）** 系统的工作流。它比传统的 RAG 更加进阶，因为它不仅利用了向量语义搜索，还结合了知识图谱（KG）来处理复杂的关系查询。

我们可以把这个架构拆分为**数据处理（索引）和用户交互（检索与生成）**两个核心阶段来理解：

1. 索引阶段：从非结构化数据到结构化知识

这一部分是图中的上半部分，负责把“死”文档变成“活”知识库。

- **文档解析 (OCR/Parsing)**：处理各种格式（PDF, CSV, Docx 等）。如果文档包含扫描件或复杂排版，会通过 OCR 技术转化为纯文本。
- **双路处理模式**（图中红色高亮区域）：这是 GraphRAG 的核心创新点：
 - **Chunking (切片)**：将文档切割成长短适中的片段，通过 Embedding 模型转化为向量，存入 **VectorStore**。这解决了“语义相似性”的检索。
 - **KG (Knowledge Graph/知识图谱)**：利用 LLM 提取文档中的实体（人、事、地）及其相互关系，形成三元组（如“A 属于 B”），存入 **Neo4j** 等图数据库。这解决了“深层关联”的检索。

2. 交互阶段：Agent 驱动的精准确问

这一部分是图中的下半部分，展示了当用户提问时，系统是如何“思考”的。

- **Agent (大脑)**：这里的 Agent 充当了调度员的角色。它接收用户的问题，并决定去哪里寻找答案。
- **混合检索 (Hybrid Retrieval)**：Agent 会同时从两个维度调取信息：
 - a. 从 **VectorStore** 找语义最相关的“片段”。
 - b. 从 **Neo4j** 找逻辑关联最强的“知识网络”。

- **Prompt 合成：** 系统将检索到的 **RAG 片段 + 知识图谱信息 + 用户的原始问题** 拼接在一起，形成一个信息量巨大的 **“完整 Prompt”**。
- **生成答案：** 最后，大模型基于这个完整的 Prompt 生成最终回答，反馈给用户。

4、技术选型

确认技术架构后，接下来就要进行技术选型了。和之前一样，我们有两种思路：一是把需求交给 ChatGPT、Gemini、DeepSeek 这类大模型，让它们直接给出推荐方案；二是凭借我们自己的技术积累来做选择。这一次，我们打算采用后一种方式，基于经验来敲定技术栈。需要重点选型的几个方向如下：

1. **向量数据库：** 选用 Milvus
2. **知识图谱：** 选用 Neo4j
3. **Agent 开发框架：** 选用 LangChain
4. **多模态数据解析：** 选用 MinerU
5. **知识图谱抽取：** 选用 LangExtract

5、技术调研

虽然我们已经完成了整体的技术选型，但对于这些组件的具体用法、细节功能以及适配性，大家可能还不够熟悉。这时我们就可以充分借助 Claude Code 来快速验证这些技术的调用方式，一边测试一边了解：这款工具能否支撑我们的项目需求，以及具体要如何实现才能满足业务场景。

5.1 LangExtract 调研

LangExtract 工具在我们整个项目中扮演着非常关键的角色，主要负责从文档中**自动抽取实体与实体间的关系**，是构建知识图谱的核心环节。

不过很多同学对它的具体用法和细节功能并不熟悉，所以接下来我们先重点验证和确认这款工具的实际能力。

代码块

```
1  ## 在Claude Code里执行
2  帮我把这个项目的源码拉取到本地文件夹：https://github.com/google/langextract
```

如下问题是我们比较关注的几个问题，我们可以直接基于源码提问。

代码块

```
1  ## 在Claude Code里执行
2  目前，我计划将 LangExtract 项目作为多模态 RAG 问答系统的数据抽取后端服务，现结合该项目源码，需对以下问题进行详细分析并给出明确答案，具体如下：
```

- 3 1. 该项目是否支持接收 PDF、DOCX 等原始文档，并完成文档解析、内容分块、知识图谱抽取、索引构建及检索的完整 GraphRAG 执行流程；
- 4 2. 该项目需接入文本模型还是多模态模型，同时是否依赖 Embedding 模型的接入才能正常运行；
- 5 3. 结合项目源码，明确其支持处理的数据格式，例如是否可直接接收 PDF、CSV 等文件作为原始输入；
- 6 4. 该项目运行过程中，是否依赖关系型数据库、向量数据库、图数据库等其他外部服务。

给出了详细的回复

刚才已经确认了输入主入口、分块逻辑、provider 和 I/O 层；现在继续补最后一块：它是否有“索引/检索/图谱库”相关实现。

Searched for 1 pattern, read 2 files (ctrl+o to expand)

不是卡住了，是我刚才在补“反证”——确认它没有索引/检索/图数据库这类实现，避免给你误导结论。现在我基于源码给你完整结论。

总结结论

LangExtract 不是一个完整的 GraphRAG 系统。
它本质上是一个“基于 LLM 的结构化信息抽取库”，核心能力是：

- 接收文本或可下载的文本型 URL
- 对长文本做分块
- 调用 LLM 做结构化抽取
- 把抽取结果映射回原文位置
- 输出为 AnnotatedDocument / JSONL / HTML 可视化

它不原生提供：

- PDF / DOCX 原始文档解析
- 多模态文档理解
- Embedding 生成
- 向量索引
- 向量检索
- 图数据库写入
- 图检索 / GraphRAG orchestration

所以如果你要把它作为“多模态 RAG 问答系统的数据抽取后端”，更准确的定位应该是：

LangExtract 适合当“文本抽取层/实体关系抽取层”组件，
但不是“原始文档解析 + Graph 建模 + 向量索引 + 检索 + 问答”的一体化后端。

1. 是否支持完整 GraphRAG 执行流程？

你问的是这条链路：

原始文档如 PDF/DOCX → 文档解析 → 分块 → 知识图谱抽取 → 索引 → 检索 → 完整 GraphRAG

我的结论是：

1.1 已支持：分块 + LLM结构化抽取

LangExtract 明确支持：

- 长文本分块
- 多轮抽取
- 抽取结果对齐回原文
- 输出结构化 extractions

证据

langextract/extraction.py:66-76

- extract() 文档说明输入是：
 - source text
 - URL 下载得到的 text
 - iterable of Document

langextract/extraction.py:338-368

- 字符串输入走 annotate_text
- 文档集合输入走 annotate_documents

langextract/chunking.py:15-20

- 明确说明库用于把大文档切成块供 text-to-text model 推理

langextract/annotation.py:222-227

- 注释写明会：
 - break documents into chunks
 - process prompts
 - perform batched inference
 - map extractions back to original document

langextract/annotation.py:237-244

- 支持 extraction_passes
- 支持 context_window_chars

1.2 部分支持：知识图谱“抽取”

它可以抽取你定义的实体、关系、属性，因此可作为知识图谱抽取前置模块。
但它只是抽取结果，不会自动帮你：

- 构建 graph schema
- 去重归一化实体
- 写入 Neo4j / Nebula / JanusGraph
- 执行图遍历或图检索

证据

README 里示例直接抽取 `character / emotion / relationship`:

README.md:63-85

以及 `Extraction` 的结构本身只是:

- `extraction_class`
- `extraction_text`
- `attributes`
- `char_interval`

见 `langextract/core/data.py:63-119`

这说明它更像一个通用 `IE/RE` 抽取器, 不是图谱存储或 `GraphRAG` 引擎。

1.3 不支持: 原始 PDF/DOCX 文档解析

源码入口只接受:

- 字符串文本
- URL 文本
- `Document` 对象
- CSV 中读取的文本列

没有看到 `PDF parser`、`DOCX parser`、`OCR`、版面分析等实现。

证据

`langextract/extraction.py:74-76`

- 输入定义是 `text / URL / iterable of Document`

`langextract/core/data.py:129-181`

- `Document` 只有 `text`, `document_id`, `additional_context`, `tokenized_text`
- 没有文件路径、二进制流、页面结构、图片对象之类字段

`langextract/io.py:50-82`

- `Dataset.load()` 只实现了 `.csv`
- 其他文件类型直接 `NotImplementedError`

`langextract/io.py:67-82`

- 只对 `.csv` 分支做处理

`langextract/io.py:261-342`

- `download_text_from_url()` 下载的是 URL 内容并尝试按文本解码
- 它检查 `Content-Type` 是否偏文本: `text/`, `application/json`, `application/xml`
- 不存在 `PDF / DOCX` 专门解析逻辑

1.4 不支持: 索引

没有发现任何 `embedding/index/vector store` 构建实现。

证据

核心抽象只有 `BaseLanguageModel.infer()`, 没有 `embedding` 接口:

`langextract/core/base_model.py:121-154`

全项目 `provider` 只有:

- `Gemini`
- `OpenAI`
- `Ollama`
- `plugin` 扩展

见 `pyproject.toml:93-97`

没有 `embedding provider`、`vector index builder`、`retriever abstraction`。

1.5 不支持: 检索

没有看到 `query → retrieval → rerank → context assembly` 的实现。

整个库的中心是 `extract`, 不是 `retrieve`。

证据

顶层公开 API 主要就是:

`langextract/__init__.py:30-48`

- `extract`
- `visualize`
- 以及若干模块暴露

没有 `search`, `retrieve`, `index`, `query`, `rag` 之类主 API。

1.6 不支持: 完整 GraphRAG 流程编排

所以完整 `GraphRAG` 链路里, `LangExtract` 只覆盖其中一段:

它覆盖的部分

- 文本分块

- 结构化抽取
- 关系抽取
- 原文 grounding / span alignment

它不覆盖的部分

- PDF/DOCX/图像解析
- OCR / layout analysis
- embedding/index
- retrieval
- graph storage
- graph query orchestration
- answer generation pipeline

2. 它接入的是文本模型还是多模态模型？是否依赖 Embedding 模型？

2.1 核心上接的是“文本生成模型”，不是多模态 pipeline

虽然支持的具体模型名里可能有本身具备多模态能力的模型，比如 gpt-4o、Gemini 某些型号，但 LangExtract 这套调用方式是文本 prompt → 文本输出。

证据

langextract/extraction.py:221-224

- 构造 PromptTemplateStructured(description=...)
- examples 也是文本例子

langextract/prompting.py:107-138

- prompt 生成完全基于文本 question / answer

langextract/providers/gemini.py:217-219

- generate_content(model=self.model_id, contents=prompt, config=config)
- contents 传入的是 prompt 字符串，不是多模态 message parts

langextract/providers/openai.py:150-158

- OpenAI 调用里 messages 的 content 也是文本 prompt
- 没有 image/audio/video part

langextract/providers/ollama.py

- 也是文本 prompt infer 路径

所以即使底层模型“可能是多模态模型”，LangExtract 当前源码使用的是文本模式。

2.2 不是原生多模态输入框架

没有看到以下任一类实现：

- 图片输入对象
- PDF page image rendering
- 多模态消息结构
- 音频转录
- 视频帧解析
- OCR 管道

因此它不是多模态文档理解框架。

2.3 不依赖 Embedding 模型

我结合源码判断：不依赖 embedding 模型，也没有 embedding 抽象。

证据

- BaseLanguageModel 只有 infer / infer_batch

- langextract/core/base_model.py:121-154

- 没有 embed()、embedding_model、vectorize() 之类接口
- pyproject.toml:30-47 依赖中没有典型向量库或 embedding 框架
- provider 也都只是生成式模型 provider，不是 embedding provider

直接结论

如果你把它接到 RAG/GraphRAG 里：

- LangExtract 需要的是生成式文本模型
- 不要求 embedding 模型
- 但如果你的整体系统要做 RAG 检索，embedding 模型仍需要由你的上层系统自己接入

3. 结合源码，它支持处理的数据格式是什么？

我按“原生支持 / 间接支持 / 不支持”分开说。

3.1 原生支持

A. 纯文本字符串

最核心输入。

证据

```
langextract/extraction.py:338-353
- if isinstance(text_or_documents, str): ... annotate_text(...)
```

B. Document 对象 / 文档对象列表

你可以自己先解析任何文件，再把文本塞进 `Document(text=...)` 交给 `LangExtract`。

证据

```
langextract/core/data.py:129-181
- Document(text, document_id, additional_context)
```

```
langextract/extraction.py:355-368
- 非字符串输入按 Iterable[data.Document] 处理
```

C. 文本型 URL

可以传 URL，`LangExtract` 自动下载文本内容再抽取。

证据

```
langextract/extraction.py:214-220
- 当 fetch_urls=True 且输入是 URL，调用 io.download_text_from_url
```

```
README.md:140-155
- 官方示例直接传 Project Gutenberg 文本 URL
```

D. CSV 文件中的文本列

库里有 `Dataset.load()`，支持从 CSV 中读出文本列和 `id` 列，生成 `Document`。

证据

```
langextract/io.py:50-82
- Dataset.load() 只支持 .csv
```

```
langextract/io.py:67-80
- 从 CSV 读取 text_key 和 id_key
```

```
langextract/io.py:191-219
- _read_csv() 使用 pandas.read_csv
```

E. JSONL 作为结果存储格式

注意这不是原始输入，而是抽取结果输入/输出格式。

证据

```
README.md:117-132
- save_annotated_documents(... output_name="extraction_results.jsonl")
- visualize("extraction_results.jsonl")
```

```
langextract/io.py:85-138
- 保存为 JSONL
```

```
langextract/io.py:140-188
- 从 JSONL 加载 AnnotatedDocument
```

3.2 间接支持 / 需要你自己预处理后再喂给它

这些格式不是 `LangExtract` 原生解析，但你可以先用别的库转成文本，再喂给 `Document(text=...)`：

A. PDF

不是原生支持。

如果你先用 `pymupdf` / `pdfplumber` / `unstructured` / `docling` 把 PDF 转文本，那么可以再交给 `LangExtract`。

为什么这么判断

- 没有 PDF parser
- `download_text_from_url()` 只做文本解码，不做 PDF 解析
- 没有 `application/pdf` 专门处理

B. DOCX

同理，不是原生支持。

C. HTML/XML

- URL 下载层面，`application/xml`、`text/*` 可以拉到文本内容
- 但它并没有 HTML DOM 提取/正文抽取逻辑
- 所以严格说是“能拿到文本字节并当文本处理”，不是“网页内容抽取框架”

证据

```
1 ... 388-389
```

langextract/10.py.288-292

- 仅把 `text/`, `application/json`, `application/xml` 视作文本型 `content-type`

D. JSON

如果是文本 JSON 内容, 你完全可以把其字符串当文本输入;
但没有 JSON schema ingestion / 文档字段抽取流水线。

3.3 不支持

下面这些从源码上看都没有原生支持:

A. PDF 原始文件解析

无 `parser` / `OCR` / `page layout`

B. DOCX 原始文件解析

无 `python-docx` 等依赖, 无 `reader`

C. Excel / XLSX

无相关依赖或 `reader`

D. PPT / PPTX

无相关依赖或 `reader`

E. Parquet

无相关依赖或 `reader`

F. 图片

无 `image input` / `multimodal messages` / `OCR`

G. 音频

无 `ASR` / `speech pipeline`

H. 视频

无视频解码 / 帧抽取 / 时间轴处理

I. 通用二进制文件上传

无 `multipart ingestion`、无 `file object` 抽象

4. 是否依赖其他外部服务, 比如关系型数据库、向量数据库或图数据库?

4.1 不依赖关系型数据库

没有看到 `MySQL` / `PostgreSQL` / `SQLite` 的运行时依赖或代码使用。

证据

`pyproject.toml`:30-47

- 没有 `sqlalchemy` / `psycopg` / `pymysql` / `sqlite adapters`

代码里也没有 `DB client` 抽象。

4.2 不依赖向量数据库

没有看到:

- `FAISS`
- `Milvus`
- `Qdrant`
- `Weaviate`
- `Pinecone`
- `Chroma`

相关依赖和调用都不存在。

证据

`pyproject.toml` 依赖清单中没有这些库

`BaseLanguageModel` 也没有 `embedding` 能力接口

4.3 不依赖图数据库

没有看到:

- `Neo4j`
- `NebulaGraph`
- `JanusGraph`
- `RDF triple store`

- graph query engine

也没有 graph persistence adapter。

4.4 运行时硬依赖：LLM 服务或本地模型服务

它的核心外部依赖是 LLM provider。

支持的 provider

Gemini

pyproject.toml:35
- google-genai>=1.39.0

pyproject.toml:95
- 内置 provider: gemini

langextract/providers/gemini.py:56-181
- 通过 Google GenAI / Vertex AI 调用

OpenAI

pyproject.toml:58-60
- optional dependency openai

pyproject.toml:97
- 内置 provider: openai

langextract/providers/openai.py:84-110
- 使用 openai.OpenAI(...)

Ollama

pyproject.toml:96
- 内置 provider: ollama

langextract/providers/ollama.py:105
- 默认本地地址 http://localhost:11434

结论

- 用 Gemini / OpenAI 时，依赖云端 API
- 用 Ollama 时，依赖本地或代理的 Ollama 服务
- 如果你写自定义 provider，也可依赖其他模型服务

4.5 可选外部依赖：GCS / Vertex AI Batch

这里有一个比较重要的“可选外部服务”：

Vertex AI Batch + GCS

如果开启 Gemini batch API，会用到：

- Vertex AI batch job
- Google Cloud Storage bucket

证据

README.md:159-163
- 支持 Vertex AI Batch API

langextract/providers/gemini.py:153-156
- 读取 batch 配置

langextract/providers/gemini.py:258-285
- 当 batch enabled 且超过阈值时调用 gemini_batch.infer_batch(...)

langextract/providers/gemini_batch.py:42-45
- imports:

- google.genai
- google.api_core.exceptions
- google.cloud.storage

langextract/providers/gemini_batch.py:79-80
- enable_caching 使用 GCS-based caching

langextract/providers/gemini_batch.py:214-245
- 自动生成/配置 GCS bucket 生命周期

结论

- 默认常规抽取不需要数据库
- 如果启用 Gemini Batch 模式，会涉及 GCS/Vertex AI

在充分了解 LangExtract 的核心功能与相关规范后，接下来我们就来编写它的使用文档，方便后续开发过程中快速查阅、规范使用。（这个非常重要）

由于 LangExtract 处于整个系统架构的中间环节，我们需要重点明确它在数据处理过程中的几项关键规范：

- 1. 处理数据时是否需要接入大模型；
- 2. 支持接入的数据数据类型；
- 3. 处理完成后输出的数据格式规范。

明确这些规范后，才能更顺畅地与上下游模块完成对接。

代码块

```
1
2  ## 在Claude Code里面执行
3  好的，我已充分理解 LangExtract 的核心功能架构。现在请帮我生成一份 LangExtract 的规范文档，具体要求如下：
4  1. 详细说明当前 LangExtract 核心 Pipeline 接收的输入规范，以及其支持的输入类型；
5  2. 明确文本模型的接入规范，以及该项目支持接入的模型类型；
6  3. 清晰说明经过 LangExtract Pipeline 处理后，输出的数据格式规范。
7
8  请创建一个 docs 文件夹，并在该文件夹下新建一个 .md 文件，将上述规范内容整理存储在该文件中。
```

```
● 这是非简单实现任务，而且会新增文档文件。我先进入计划模式，给你一个精简执行方案后直接落地。
● Entered plan mode
  Claude is now exploring and designing an implementation approach.
● Bash(cd "D:/code/GraphRAG-Agent/langextract" && pwd)
  └─ /d/code/GraphRAG-Agent/langextract
● Explore(Explore doc patterns) Sonnet 4.6
  └─ Read(.claude/worktrees/agent-ad53ebcf\README.md)
    Read(.claude/worktrees/agent-ad53ebcf\README.md · pages 1-20)
    Bash(ls "/d/code/GraphRAG-Agent/langextract/.claude/worktrees/agent-ad53ebcf/docs" && ls
      "/d/code/GraphRAG-Agent/langextract/.claude/worktrees/agent-ad53ebcf/langext...)
    Running...
    +1 more tool use (ctrl+o to expand)
    (ctrl+b to run in background)
· 编写 LangExtract 规范文档.. (3m 18s · ↓ 858 tokens)
  └─ ■ 编写 LangExtract 规范文档

> █

Model: Sonnet 4.6 (1M context) | Ctx: 120.9k | ↵ main | (+0,-0)
plan mode on (shift+tab to cycle)
```

生成文档

• 已创建文档：

- D:\code\GraphRAG-Agent\langextract\docs\extraction_contract.md

内容已覆盖你要求的 3 部分：

1. 输入规范：主入口参数、支持的输入类型、CSV / URL / Document / few-shot 示例规范、以及不支持的原始文档类型边界
2. 模型接入规范：Provider 机制、Gemini / OpenAI / Ollama 支持情况、文本模型接入方式、是否依赖 embedding
3. 输出格式规范：AnnotatedDocument、Extraction、CharInterval、JSONL 持久化格式与 grounding 说明

如果你愿意，我下一步可以继续帮你做两件事里的任意一个：

- 把这份文档再整理成更偏“对外接口说明书”风格
- 或者继续补一份“LangExtract 接入 GraphRAG-Agent 的集成设计文档”

* Brewed for 14m 21s

langextract-extraction-contract

> █

Model: Sonnet 4.6 (1M context) | Ctx: 0 | `main` | (+0,-0)
▶ accept edits on (shift+tab to cycle)

```
extraction_contract.md x
langextract > docs > extraction_contract.md > # LangExtract Pipeline 规范文档
1 # LangExtract Pipeline 规范文档
2
3 本文档基于当前仓库源码整理，目的是明确说明 LangExtract 的三类核心契约：
4
5 1. Pipeline 接收的输入规范与支持的输入类型
6 2. 文本模型 / Provider 的接入规范与支持的模型类型
7 3. 经过 LangExtract Pipeline 输出的数据格式规范
8
9 > 说明：本文档描述的是 **当前源码已实现的能力**，不将外部可扩展能力视为内建能力。
10
11 ---
12
13 ## 1. LangExtract 核心 Pipeline 概述
14
15 LangExtract 的核心能力是：
16
17 - 接收文本输入或文本文档对象
18 - 将长文本切分为多个 chunk
19 - 基于 few-shot prompt 调用 LLM 做结构化抽取
20 - 将抽取结果对齐回原始文本位置
21 - 返回结构化的 `AnnotatedDocument`
22 - 支持将结果保存为 JSONL，并生成 HTML 可视化
23
24 核心入口函数为：
25
26 - `langextract/extraction.py:36` 的 `extract(...)`
27
28 核心处理链路可概括为：
```

5.2 MinerU 调研

接下来，我们来调研 LangExtract 的上游工具——MinerU。MinerU 是一款开源项目，提供了多种灵活的使用方式，其中一种便捷的方式就是支持在线调用，其官方在线使用文档可参考：<https://mineru.net/apiManage/docs>。接下来，我们先让 Claude Code 帮我们梳理一下它的具体使用方法，方便我们后续快速上手调用。

代码块

- 1 *#在Claude Code中输入*
- 2 现在我需要基于 MinerU 项目实现文档解析需求，其官方文档地址为：<https://mineru.net/apiManage/docs>。请详细研读这份官方文档，明确 MinerU 支持解析的原始文档数据格式，以及对应的输出数据格式分别是什么。

现在我需要基于 MinerU 项目实现文档解析需求，图片里就是他的docs文档，请详细研读这份官方文档，明确 MinerU 支持解析的原始文档数据格式，以及对应的输出数据格式分别是什么。

掌握了基本信息以后，继续梳理

代码块

- 1 *# 在Claude Code中继续输入*
- 2 我现在要开始进行 MinerU 文档解析的 MVP 测试，请结合你掌握的信息，明确说明在执行该 MVP 测试时，是否需要配置 API_Key 等外部配置文件，并进行详细梳理。

接下来生成 MinerU 的开发规范文档

代码块

- 1 *## Claude Code中执行*
- 2 请结合上述调研结果，帮我生成一份 MinerU 相关的规范文档，具体要求如下：
- 3 1. 明确 MinerU 支持的原始输入文件格式；
- 4 2. 本次采用云端 API 方案，需详细说明文档解析后的输出格式、最终生成的各类文件，以及每个文件的具体字段说明；
- 5 3. 梳理文档解析后生成的布局信息，为后续精准提取数值提供支撑；
- 6 4. 明确执行 MVP 测试所需的必要字段、必传参数。
- 7 请在 docs 文件夹下新建一个 .md 文件，将上述所有规范内容整理并存储在该文件中。



```
1 # MinerU 云端 API 输入输出规范与 MVP 必要字段说明
2
3 > 本文基于 MinerU 官方 API 文档整理，面向云端 API 接入场景，重点说明输入文件格式、输出产物、布局信息以及 MVP 测试必需字段。
4
5 ## 1. 支持的原始输入文件格式
6
7 MinerU 云端 API 分为两类：
8
9 - **精准解析 API**：适合高精度、结构化提取、批量处理、HTML 解析场景。
10 - **Agent 轻量解析 API**：适合快速跑通 MVP、只拿 Markdown 结果的场景。
11
12 ### 1.1 精准解析 API 支持的输入格式
13
14 支持通过远程 URL 或本地文件上传后解析。
15
16 #### 支持的格式
17 - `pdf`
18 - 图片：`png`、`jpg`、`jpeg`、`jp2`、`webp`、`gif`、`bmp`
19 - Word：`doc`、`docx`
20 - PowerPoint：`ppt`、`pptx`
21 - `html`
22
23 #### 限制
24 - 单文件大小：`<= 200MB`
25 - 单文件页数：`<= 600 页`
26 - 批量文件数：`<= 200`
27
28 #### 特殊说明
```

6、MVP测试

6.1 MinerU MVP 测试

在完成规范文档编写后，我们将依据这份规范开展MVP测试。首先针对MinerU进行验证：传入PDF文件作为输入，由MinerU完成文档解析并输出对应的解析结果。

在使用之前，大家需要先前往对应平台申请 Token，即可免费进行调用。(<https://mineru.net/>)

6t1Lq9hdW9GwEeXFwmGrgRwkl9XLUqBLwlYEbq5OZZy00Yk4pYjBGh0nhuKQsrceedzTUOuXWmUhK1sOyDGBYw

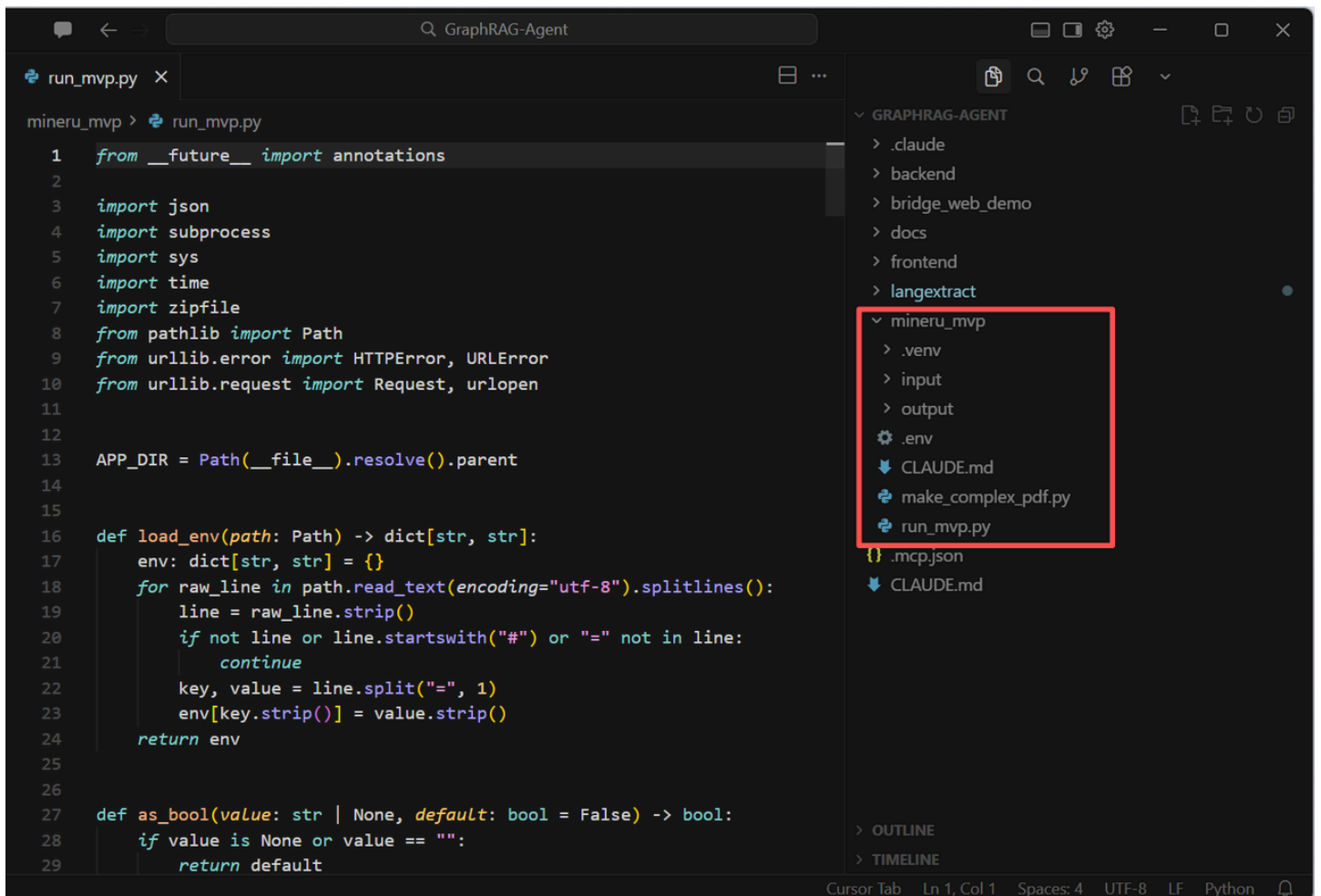
- 4 3. 在测试文件夹内新建一个示例 PDF 文件，用于模拟本地 PDF 输入。
- 5 4. 基于以上要求，编写完整可运行的 Pipeline 代码，实现：
- 6 本地 PDF 文件加载
- 7 调用 MinerU 云端 API 进行解析
- 8 解析结果保存到本地
- 9 5. 代码完成后，请给出**本地运行步骤**和**预期执行结果说明**。

请参考规范文档 @docs/MinerU_MVP测试配置指南.md，基于 MinerU 云端 API 规范生成一套可直接运行的 **MVP 测试代码及完整流程**，具体要求如下：

1. 创建一个独立文件夹，用于存放所有 MinerU MVP 测试相关代码。
2. 将所有配置信息（包括 API 地址、Token 等）统一写入 .env 环境变量文件中；我已提供可用 Token：

eyJ0eXBlljoiSldUliwiYWxnljoiSFM1MTIifQ.eyJqdGkiOiIxNjkwMDY3MSIsInJvbCI6IlJPTEVfUkVHSVNURVliLCJpc3MiOiJPcGVuWExhYiIsImIhdCI6MTc3NDQ1MTc0NCwiY2xpZW50SWQiOiJsa3pkeDU3bnZ5MjJqa3BxOXgydyIsInBob25lljoiIiwib3BlbklkljpudWxsLCJ1dWlkIjoiYmYzODhjMGQtZWYwZS00ZDQzLWJmZWEtYTNhMWRkMzJhMmJiliwiZW1haWwiOiIiLCJleHAiOiJlE3ODIyMjc3NDR9.6t1Lq9hdW9GwEeXFwmGrgRwkl9XLUqBLwlYEbq5OZZy00Yk4pYjBGh0nhuKQsrceedzTUOuXWmUhK1sOyDGBYw

3. 在测试文件夹内新建一个示例 PDF 文件，用于模拟本地 PDF 输入。
4. 基于以上要求，编写完整可运行的 Pipeline 代码，实现：
- 本地 PDF 文件加载
- 调用 MinerU 云端 API 进行解析
- 解析结果保存到本地
5. 代码完成后，请给出**本地运行步骤**和**预期执行结果说明**。



创建更复杂的PDF文件，测试MVP代码！

> 很好，再创建要给复杂一点的PDF文件，比如里面包含表格，然后再测试一下代码，并给我说测试结果。

• 我先生成一个更复杂、包含表格的 PDF。

```
• Write(mineru_mvp\make_complex_pdf.py)
  | Wrote 103 lines to mineru_mvp\make_complex_pdf.py
    1 from pathlib import Path
    2
    3 OUT = Path(r"D:/code/GraphRAG-Agent/mineru_mvp/input/complex_table.pdf")
    4
    5
    6 def esc(text: str) -> bytes:
    7     return text.replace('\\', '\\\\').replace('(', '\\(').replace(')', '\\)').encode('latin-1', errors='
      replace')
    8
    9 ops = []
   10 ops.append(b"BT")
   ... +93 lines (ctrl+o to expand)
```

• 我先运行脚本生成复杂 PDF。

```
• Bash("D:/soft/soft/anaconda/python.exe" "D:/code/GraphRAG-Agent/mineru_mvp/make_complex_pdf.py")
  | D:\code\GraphRAG-Agent\mineru_mvp\input\complex_table.pdf
```

• 我把 .env 的输入文件切到这个复杂 PDF。

```
• Update(mineru_mvp\.env)
  | Added 1 line, removed 1 line
    8 MINERU_IS_OCR=false
    9 MINERU_EXTRA_FORMATS=
   10 MINERU_DATA_ID=mineru-mvp-local-sample
   11 -MINERU_INPUT_FILE=./input/sample.pdf
   11 +MINERU_INPUT_FILE=./input/complex_table.pdf
   12 MINERU_OUTPUT_DIR=./output
   13 MINERU_POLL_INTERVAL_SECONDS=5
   14 MINERU_POLL_TIMEOUT_SECONDS=300
```

• 我开始重新跑复杂 PDF 的 MinerU 测试。

```
• Bash("D:/soft/soft/anaconda/python.exe" "D:/code/GraphRAG-Agent/mineru_mvp/run_mvp.py")
  | [poll] batch_id=29a7e1e8-ee4c-4be6-829a-076bfalbe873 state=waiting-file
  | [poll] batch_id=29a7e1e8-ee4c-4be6-829a-076bfalbe873 state=done
  | {
  | ... +21 lines (ctrl+o to expand)
  | (timeout 10m)
```

• 我再读取复杂 PDF 的 Markdown 结果，确认表格解析效果。

Read 1 file (ctrl+o to expand)

• 我再看一下 content_list_v2.json，确认表格内容有没有被结构化出来。

Read 1 file (ctrl+o to expand)

• 已完成第二轮复杂 PDF 测试，而且结果是成功的。

我做了什么

1. 新建了一个更复杂的 PDF

文件位置：

mineru_mvp/input/complex_table.pdf

这个 PDF 包含：

- 标题区
- 说明文字
- 一个 4 列 5 行的表格
- 表格数据包括数值和百分比
- 表格下方还有摘要列表

表格内容大致是：

Metric	Q1	Q2	Q3
Revenue	12345.67	15200.00	16789.30
Cost	8450.10	9022.45	9988.80
Profit	3895.57	6177.55	6800.50
Margin	31.55%	40.64%	40.51%

2. 更新了 .env

我调整了输入文件路径

我把输入文件切换成：

```
MINERU_INPUT_FILE=./input/complex_table.pdf
```

3. 重新执行了完整 MinerU Cloud Pipeline

还是用你指定的 Python：

```
D:/soft/soft/anaconda/python.exe
```

执行结果：成功

本次测试结果

batch_id

29a7e1e8-ee4c-4be6-829a-076bfa1be873

任务状态

done

输出目录

mineru_mvp/output/29a7e1e8-ee4c-4be6-829a-076bfa1be873/

本次生成的结果文件

ZIP 解压后包含：

- full.md
- layout.json
- content_list_v2.json
- da6ee564-a49a-4719-a3d4-d6c96b5a3e35_model.json
- da6ee564-a49a-4719-a3d4-d6c96b5a3e35_content_list.json
- da6ee564-a49a-4719-a3d4-d6c96b5a3e35_origin.pdf
- images/...jpg

这说明：

- 云端解析成功
- 表格结果文件成功返回
- 布局和结构化内容文件也都成功返回

解析效果验证

1. full.md 的结果

我读取了 full.md，内容是：

- 一张表格截图图片引用
- Highlights: 标题
- 三条列表说明

也就是说：

Markdown 主结果里没有直接把表格展开成 Markdown 表格，而是把表格以图片形式引用了。

这点很重要：

- 如果你只看 full.md，表格数值不够直接
- 但这不代表 MinerU 没识别出来

2. content_list_v2.json 的结果

这里效果很好。

我读取到的结构化内容里，有一个明确的：

- type: "table"

并且它的 content.html 中已经结构化出了完整表格：

```
<table>
<tr><td>Metric</td><td>Q1</td><td>Q2</td><td>Q3</td></tr>
<tr><td>Revenue</td><td>12345.67</td><td>15200.00</td><td>16789.30</td></tr>
<tr><td>Cost</td><td>8450.10</td><td>9022.45</td><td>9988.80</td></tr>
<tr><td>Profit</td><td>3895.57</td><td>6177.55</td><td>6800.50</td></tr>
<tr><td>Margin</td><td>31.55%</td><td>40.64%</td><td>40.51%</td></tr>
</table>
```

同时它还给出了：

- table_type: "simple_table"
- table_nest_level: 1
- bbox: [114, 138, 936, 343]

这说明：

关键结论

对于复杂 PDF 中的表格：

- MinerU 成功识别了表格
- MinerU 成功提取了表格中的所有数值
- MinerU 成功返回了结构化 HTML 表格
- MinerU 成功返回了表格的 bbox 布局坐标

对你后续“精准提取数值”的意义

这次测试非常有价值，因为它验证了你最关心的能力：

已验证成功的点

1. 本地复杂 PDF 能上传
2. 云端精准解析 API 能跑通
3. 表格能被识别成结构化 table block
4. 数值能进入结构化内容
5. 表格有 bbox，可做布局定位
6. 标题、列表、页眉也能分块识别

这意味着你后续可以这样做

你不应该只依赖 full.md，而应该优先用：

- content_list_v2.json
- layout.json

特别是：

- 表格数值提取：优先读 content_list_v2.json
- 位置信息/空间对齐：结合 bbox
- 复杂页面归属判断：再结合 layout.json

简单结论

本次复杂 PDF 测试结果

成功。

成功级别

不是仅仅“接口跑通”，而是已经验证到：

- 表格识别成功
- 表格数值提取成功
- 结构化输出成功
- 布局信息返回成功

这说明你现在这个 MVP Pipeline 已经具备继续往“自动数值抽取”方向扩展的基础。

当前你已经有的关键文件

复杂 PDF 输入

mineru_mvp/input/complex_table.pdf

本次测试输出

mineru_mvp/output/29a7e1e8-ee4c-4be6-829a-076bfa1be873/

最值得看的文件

- extracted/content_list_v2.json
- extracted/layout.json
- extracted/full.md

如果你愿意，我下一步可以继续直接帮你做：

1. 写一个表格数值抽取脚本
 - 从 content_list_v2.json 中抽出表格
 - 转成 pandas 风格 JSON / CSV
2. 写一个布局解析脚本
 - 读取 layout.json
 - 输出 block bbox 和类型
3. 把整个 MVP 扩展成“输入 PDF -> 输出结构化 JSON”
 - 直接为 GraphRAG 后续流程准备数据

如果你要，我建议下一步我直接做第 1 个：
把 MinerU 输出的表格内容自动提取成结构化 JSON。

基于已生成的 MVP 测试代码，进一步优化完善，生成一套更规范、更具通用性的 MinerU 开发文档，明确开发流程、代码规范及使用标准，为后续正式开发、维护及协作提供参考依据。

代码块

- ```
1 ## Claude Code中执行
2 现在请结合 MinerU API 的实际执行流程、最终输出结果，参考其官方文档
 (https://mineru.net/apiManage/docs)，对 @docs\mineru_cloud_api_io_spec.md 文件
 进行更新完善，具体要求如下：
3 1. 补充完整的 MinerU Pipeline 执行思路，并明确实际测试脚本的存放路径；
4 2. 读取本地实际生成的解析文档，与官方文档进行对比校验，若存在内容冲突，以本地实际文档的输出规范为准，同步更新该 .md 文件；
5 3. 详细解析 MinerU Pipeline 执行过程中的关键参数规范，明确各参数的含义、取值范围及使用要求；
6 4. 梳理本地实际生成的关键文件，与官方文档中提及的文件类型、数量进行对比，若存在冲突，以本地实际生成的文件为准进行更新，最终生成新版本文档 mineru_cloud_api_io_spec_v1.0.md。
```

现在请结合 MinerU API 的实际执行流程、最终输出结果，对 @docs\MinerU\_MVP测试配置指南.md 文件进行更新完善，具体要求如下：

1. 补充完整的 MinerU Pipeline 执行思路，并明确实际测试脚本的存放路径；
2. 读取本地实际生成的解析文档，与官方文档进行对比校验，若存在内容冲突，以本地实际文档的输出规范为准，同步更新该 .md 文件；
3. 详细解析 MinerU Pipeline 执行过程中的关键参数规范，明确各参数的含义、取值范围及使用要求；
4. 梳理本地实际生成的关键文件，与官方文档中提及的文件类型、数量进行对比，若存在冲突，以本地实际生成的文件为准进行更新，最终生成新版本文档 MinerU\_MVP测试配置指南.\_v1.0.md。

• 已完成，生成文件：

- docs/mineru\_cloud\_api\_io\_spec\_v1.0.md

这版已按你的 4 点要求更新，核心变化是：

- 补充了本仓库实际跑通的 MinerU Pipeline 执行流程
- 标明了实际测试脚本位置
  - mineru\_mvp/run\_mvp.py
  - mineru\_mvp/make\_complex\_pdf.py
- 依据本地真实输出修正了规范，明确：
  - 当前仓库实际跑通的是 精准解析 API + 本地文件上传 batch 模式
  - 实际查询主键应按 `batch_id + extract_result[]` 读取
  - 本地产物除旧文档里的 `full.md / layout.json / *_model.json / *_content_list.json` 外，还实测存在：
    - `content_list_v2.json`
    - `*_origin.pdf`
    - `images/`
- 增补了 MinerU Pipeline 关键参数规范，并对照 `.env` 与 `run_mvp.py` 的真实字段映射进行了说明

另外我也按“本地结果优先”修正了几个关键冲突点：

- 旧文档偏单任务字段：
  - `data.task_id`
  - `data.state`
  - `data.full_zip_url`
- 本仓库实测应按 batch 结果读取：
  - `data.batch_id`
  - `data.extract_result[].state`
  - `data.extract_result[].full_zip_url`

如果你要，我下一步可以继续直接帮你把 `docs/mineru_cloud_api_io_spec.md` 也原地覆盖升级成同样内容。

\* Baked for 9m 29s

>

Model: Sonnet 4.6 (1M context) | Ctx: 0 | ~~no~~ git | (no git)

## 6.2 解决 MinerU 依赖问题

在测试好 MinerU MVP 之后，接入 LangExtract 之前需要考虑：不同服务组件的外部环境依赖兼容问题。我们给当前 MinerU MVP 项目编写一个子 CLAUDE.md 文件。

在 **Claude Code** 中，`CLAUDE.md` 文件的作用类似于给 AI 助手的一份“项目行动指南”或“上下文记忆本”。

它旨在通过明确的指令和规则，减少 Claude 在协助你编写代码时的反复确认，提高开发效率。其核心作用可以概括为以下几点：

### 1. 明确开发规范 (Style Guide)

你可以规定项目中必须遵守的代码风格，例如：

- **缩进与格式**：使用 2 空格还是 4 空格。
- **命名约定**：变量名使用 `camelCase` 还是 `snake_case`。
- **特定偏好**：例如“优先使用函数式组件而非类组件”，“禁止使用 `any` 类型”。

### 2. 存储项目上下文 (Context Retention)

对于大型项目，Claude 可能无法一次性理解所有结构。`CLAUDE.md` 可以记录：

- **技术栈**：本项目使用的是 Python 3.10 + Fast API。
- **目录结构**：关键业务逻辑位于哪个文件夹。

- **运行/测试命令**：例如“使用 `pytest` 运行测试”，“使用 `make build` 进行编译”。

### 3. 定义任务流程 (Workflow)

就像我们目前遇到的问题，在 **MinerU** 和 **LangExtract** 之间处理环境依赖。我们可以在文件中写入：

“在添加新功能前，必须先更新依赖配置文件，并确保 `pip install -r requirements.txt` 能正常通过。”

### 4. 减少“幻觉”与重复错误

如果你发现 Claude 反复犯同一个错误（比如用了某个已废弃的 API），你可以直接在 `CLAUDE.md` 里加一条：

“**注意**：不要使用 `OldLibrary.v1`，本项目已全面迁移至 `NewLibrary.v2`。”

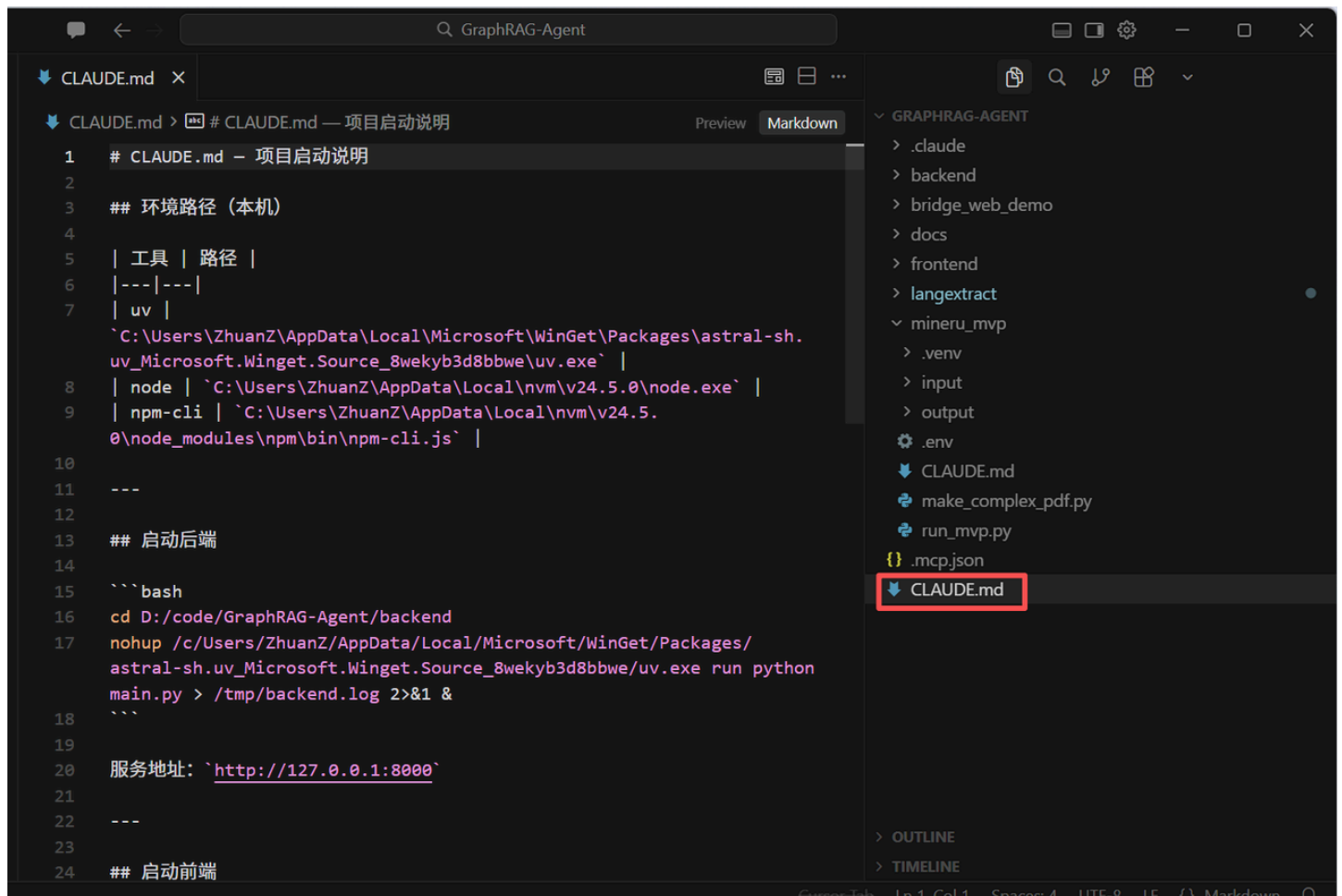
代码块

- ```
1  ## 在Claude Code中执行
2  在将 MinerU 的输出接入 LangExtract 之前，为避免不同项目组件之间因环境依赖冲突造成环境污染，需先进行环境隔离配置。请在路径 `D:\code\GraphRAG-Agent\mineru_mvp` 下，使用 uv 工具新建一个虚拟环境，并完成以下两项配置：
3  1. 参照 Claude Code 官方规范，编写一份 CLAUDE.md 文件，明确说明：MinerU 组件启动前，必须先进入该新建的子虚拟环境，再执行相关运行操作；
4  2. 更新文档 `@D:\code\GraphRAG-Agent\docs\mineru_cloud_api_io_spec_v1.0.md`，在 Pipeline 启动说明部分，补充虚拟环境的切换步骤及相关注意事项，确保后续执行流程规范统一。
```

在将 MinerU 的输出接入 LangExtract 之前，为避免不同项目组件之间因环境依赖冲突造成环境污染，需先进行环境隔离配置。请在路径 `E:\大模型课程录制\Vibe Coding 全栈开发实战课\篇章八：Claude Code 案例实践\code\GraphRAG-Agent\mineru\_mvp\_test` 下，使用 uv 工具新建一个虚拟环境，并完成以下两项配置：

1. 参照 Claude Code 官方规范，编写一份 CLAUDE.md 文件，明确说明：MinerU 组件启动前，必须先进入该新建的子虚拟环境，再执行相关运行操作；
2. 更新文档 `@E:\大模型课程录制\Vibe Coding 全栈开发实战课\篇章八：Claude Code 案例实践\code\GraphRAG-Agent\docs\MinerU\_MVP测试配置指南.\_v1.0.md`，在 Pipeline 启动说明部分，补充虚拟环境的切换步骤及相关注意事项，确保后续执行流程规范统一。

生成了CLAUDE.md文件



6.3 LangExtract MVP 测试

接下来，我们将针对 LangExtract 组件开展 MVP 测试，验证其接收 MinerU 解析输出、完成后续数据处理的核心功能是否正常可用。

1) 安装依赖

代码块

- 1 `## Claude Code中执行`
- 2 现在我对 LangExtract 组件开展项目功能测试，麻烦你在路径 GraphRAG-Agent\langextract_src 下，使用 uv 工具新建一个虚拟环境，并完成所有外部服务依赖的安裝配置，为后续功能测试做好环境准备。

2) 确认配置信息

代码块

- 1 请查看文档 D:\code\GraphRAG-Agent\langextract\docs\extraction_contract.md，根据其中规范确认：若要开展 LangExtract 最小化 MVP 测试，需要准备哪些必要的配置信息（如 API Key 等）。

请查看文档 E:\大模型课程录制\Vibe Coding 全栈开发实战课\篇章八：Claude Code 案例实践\code\GraphRAG-Agent\docs\LangExtract_规范文档.md，根据其中规范确认：若要开展 LangExtract 最小化 MVP 测试，需要准备哪些必要的配置信息（如 API Key 等）。

最终建议：你要做“最小化 MVP”，分两种最优路径

路径 1：最快云端 MVP

准备：

- LANGEXTRACT_API_KEY 或 GEMINI_API_KEY
- model_id="gemini-2.5-flash"
- 1 段输入文本
- 1 条 prompt_description
- 至少 1 个 ExampleData

路径 2：最快本地 MVP

准备：

- 本地 Ollama 已启动
- 本地已有模型，如 gemma2:2b
- model_url="http://localhost:11434"
- 1 段输入文本
- 1 条 prompt_description
- 至少 1 个 ExampleData

如果你要，我下一步可以直接帮你输出一份 LangExtract 最小 MVP 配置清单，我会按：
1. Gemini 版
2. Ollama 版
各给你一套可直接运行的最小示例。

了解到测试需要用到模型，那我们就注册并申请一个 DeepSeek 的 API Key 来使用。（到 DeepSeek 官网注册即可）

我申请到的 Key 如下：

代码块

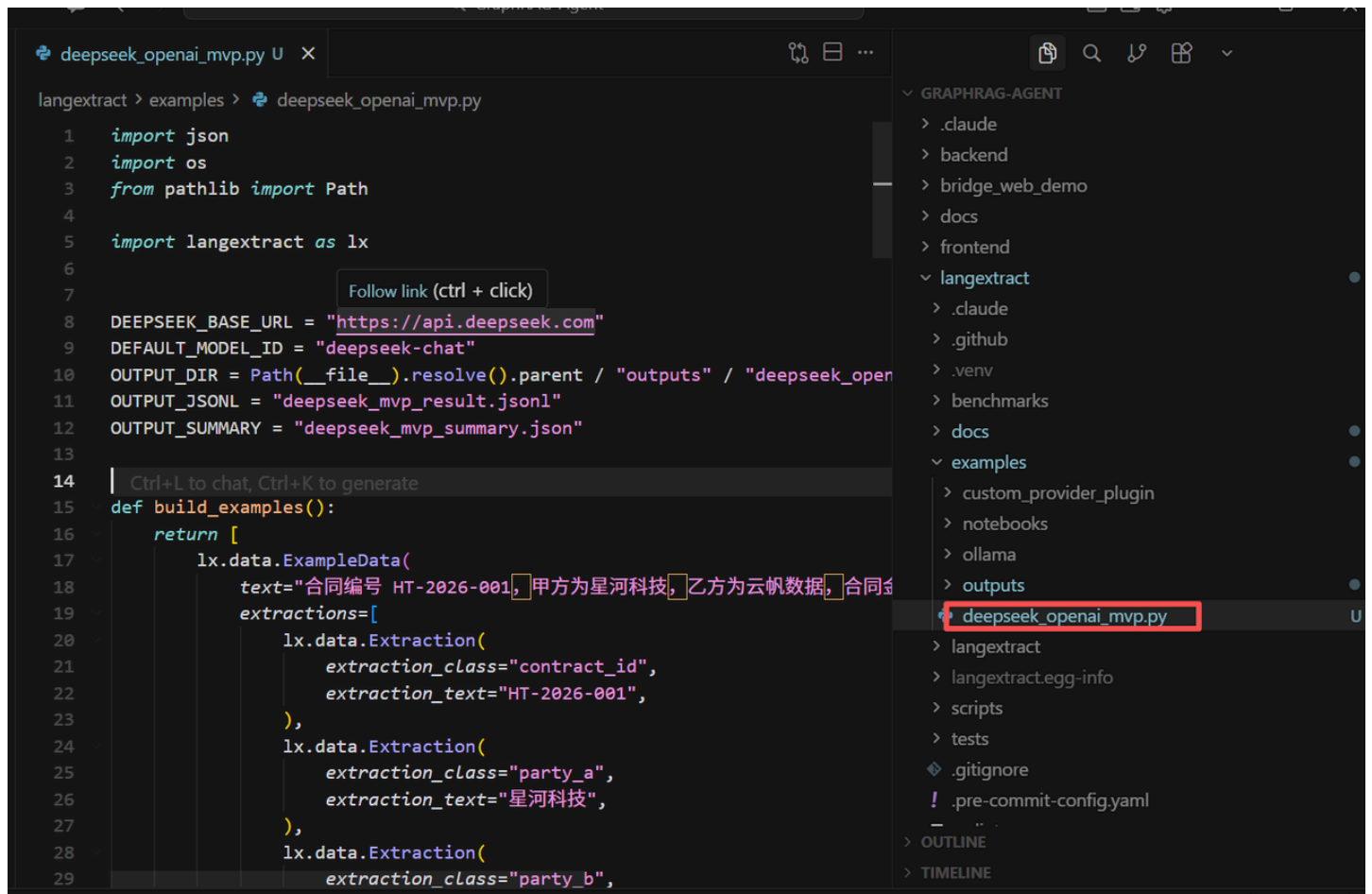
```
1 sk-1d47dcba4cbb4be4beebf23ef12932f0
```

接下来，请 Claude Code 参照此前制定的 LangExtract 开发规范文档，生成该组件的 MVP 测试代码，并完成代码的运行测试，验证核心功能是否正常。

代码块

- 1 我将通过 OpenAI Provider 接入 DeepSeek 模型，相关官方文档如下：https://api-docs.deepseek.com/zh-cn/guides/thinking_mode，https://api-docs.deepseek.com/zh-cn/guides/tool_calls
- 2 我的 API Key 为：sk-1d47dcba4cbb4be4beebf23ef12932f0

- 3 请帮我完成 MVP 流程验证，具体要求如下：
- 4 1. 严格遵循 E:\大模型课程录制\Vibe Coding 全栈开发实战课\篇章八：Claude Code 案例实践\code\GraphRAG-Agent\docs\LangExtract_规范文档.md 规范，完成 DeepSeek API Key 的接入配置；
- 5 2. 实现完整的信息提取链路：自行构造模拟文本数据 → 调用模型进行解析 → 生成结构化提取结果；
- 6 3. 代码完成后在本地运行测试，并反馈最终执行结果。



The screenshot shows a code editor with a Python file named `deepseek_openai_mvp.py`. The code defines constants for the DeepSeek API URL, model ID, and output paths. It then defines a `build_examples()` function that returns a list of `lx.data.ExampleData` objects. Each object contains a `text` string and a list of `lx.data.Extraction` objects. The first extraction is for the `contract_id` with the value `HT-2026-001`. The second and third extractions are for `party_a` and `party_b` with the value `星河科技`.

```
1 import json
2 import os
3 from pathlib import Path
4
5 import langextract as lx
6
7
8 DEEPSEEK_BASE_URL = "https://api.deepseek.com"
9 DEFAULT_MODEL_ID = "deepseek-chat"
10 OUTPUT_DIR = Path(__file__).resolve().parent / "outputs" / "deepseek_openai_mvp"
11 OUTPUT_JSONL = "deepseek_mvp_result.jsonl"
12 OUTPUT_SUMMARY = "deepseek_mvp_summary.json"
13
14
15 def build_examples():
16     return [
17         lx.data.ExampleData(
18             text="合同编号 HT-2026-001, 甲方为星河科技, 乙方为云帆数据, 合同金额 1000000 元。",
19             extractions=[
20                 lx.data.Extraction(
21                     extraction_class="contract_id",
22                     extraction_text="HT-2026-001",
23                 ),
24                 lx.data.Extraction(
25                     extraction_class="party_a",
26                     extraction_text="星河科技",
27                 ),
28                 lx.data.Extraction(
29                     extraction_class="party_b",
30                     extraction_text="云帆数据",
31                 ),
32             ],
33         )
34     ]
```

The file explorer on the right shows the project structure, with the `deepseek_openai_mvp.py` file highlighted in the `examples` directory.

结合本次 LangExtract 组件的 MVP 测试结果，对其开发规范文档进行补充、更新与完善，确保规范内容与实际测试场景、执行流程保持一致，为后续正式开发提供准确依据。

代码块

- 1 现在请结合你实际的执行流程、LangExtract Pipeline 的最终输出结果，对相关文档进行更新完善，具体要求如下：
- 2 1. 补充完整的 LangExtract Pipeline 执行思路，并明确实际测试脚本的存放路径；
- 3 2. 读取本地实际生成的文档，以本地实际的文档输出规范为唯一标准，同步更新对应 .md 文件内容；
- 4 3. 详细解析 LangExtract Pipeline 执行过程中的关键参数规范，明确各参数的含义、取值及使用要求；
- 5 4. 梳理本地实际生成的关键文件，以本地实际的文件数量、类型为依据，更新 .md 文件中的相关描述；
- 6 5. 参照 Claude Code 官方规范，编写一份 CLAUDE.md 文件，明确说明：LangExtract 组件启动前，必须先进入子虚拟环境，再执行相关运行操作；

现在请结合你实际的执行流程、LangExtract Pipeline 的最终输出结果，对相关文档进行更新完善，具体要求如下：

1. 补充完整的 LangExtract Pipeline 执行思路，并明确实际测试脚本的存放路径；
2. 读取本地实际生成的文档，以本地实际的文档输出规范为唯一标准，同步更新对应 .md 文件内容；
3. 详细解析 LangExtract Pipeline 执行过程中的关键参数规范，明确各参数的含义、取值及使用要求；
4. 梳理本地实际生成的关键文件，以本地实际的文件数量、类型为依据，更新 .md 文件中的相关描述；
5. 参照 Claude Code 官方规范，编写一份 CLAUDE.md 文件，明确说明：LangExtract 组件启动前，必须先进入子虚拟环境，再执行相关运行操作；
6. 完成上述所有更新后，最终生成新本文档 LangExtract_规范文档-v1.0.md。

6.4 MinerU 和 LangExtract 对接 MVP 测试



因为有了前面的开发规范，接下来的流程就相对容易多了。

代码块

```
1  ## Claude Code中执行
2  请严格参照以下两个规范文档：F:\GraphRAGAgent\docs\mineru_specification-v1.0.md
   F:\GraphRAGAgent\docs\Langextract_specification-v1.0.md
3  我需要将 MinerU 解析完成后的文档，传入 LangExtract 组件，进而完成知识图谱的生成。请你依据这两个文档中明确的输入、输出规范，先为我制定一份完整的组件对接规划方案，确保 MinerU 输出与 LangExtract 输入无缝衔接、流程规范可控。
```

请严格参照以下两个规范文档：docs\LangExtract_规范文档-v1.0.md docs\MinerU_MVP测试配置指南_v1.0.md

我需要将 MinerU 解析完成后的文档，传入 LangExtract 组件，进而完成知识图谱的生成。请你依据这两个文档中明确的输入、输出规范，先为我制定一份完整的组件对接规划方案，确保 MinerU 输出与 LangExtract 输入无缝衔接、流程规范可控。

完成GrahphRAG索引规范的文档。

代码块

```
1  ## Claude Code
```

- 2 现在请结合你实际的执行流程，以及 MinerU 与 LangExtract 组件的最终输出结果（该流程将作为 GraphRAG 索引阶段的核心流程），请按以下要求完成相关处理与文档编写：
- 3 1. 补充完整的 BridgePipeline 执行思路，并明确实际测试脚本的具体存放位置；
- 4 2. 读取本地实际生成的文档，以本地实际的文档输出规范为唯一标准，同步更新对应 .md 文件内容；
- 5 3. 详细解析 MinerU Pipeline 与 LangExtract Pipeline 各自的关键参数规范，明确各参数的含义、取值范围及使用要求；
- 6 4. 详细说明 MinerU 与 LangExtract 两个组件的接口对接规范，包括数据格式、传输方式、衔接逻辑等，确保组件间无缝对接；
- 7 5. 详细明确 BridgePipeline 的最终输出关键参数规范，清晰定义各输出参数的类型、含义及使用场景；
- 8 6. 参照 Claude Code 官方规范，编写一份 CLAUDE.md 文件，明确说明：只要启动 BridgePipeline 组件，必须先进入子虚拟环境，再执行相关运行操作。
- 9
- 10 请在 docs 文件夹中新建一份 xxx-v1.0.md 文件（xxx 可结合流程特性命名，如 bridge-pipeline-specification-v1.0.md），将上述所有内容统一整理存储其中，确保文档规范、逻辑清晰，可作为后续 GraphRAG 索引阶段的执行依据。

现在请结合你实际的执行流程，以及 MinerU 与 LangExtract 组件的最终输出结果（该流程将作为 GraphRAG 索引阶段的核心流程），请按以下要求完成相关处理与文档编写：

1. 补充完整的 BridgePipeline 执行思路，并明确实际测试脚本的具体存放位置；
2. 读取本地实际生成的文档，以本地实际的文档输出规范为唯一标准，同步更新对应 .md 文件内容；
3. 详细解析 MinerU Pipeline 与 LangExtract Pipeline 各自的关键参数规范，明确各参数的含义、取值范围及使用要求；
4. 详细说明 MinerU 与 LangExtract 两个组件的接口对接规范，包括数据格式、传输方式、衔接逻辑等，确保组件间无缝对接；
5. 详细明确 BridgePipeline 的最终输出关键参数规范，清晰定义各输出参数的类型、含义及使用场景；
6. 参照 Claude Code 官方规范，编写一份 CLAUDE.md 文件，明确说明：只要启动 BridgePipeline 组件，必须先进入子虚拟环境，再执行相关运行操作。

请在 docs 文件夹中新建一份 xxx-v1.0.md 文件（xxx 可结合流程特性命名，如 bridge-pipeline-specification-v1.0.md），将上述所有内容统一整理存储其中，确保文档规范、逻辑清晰，可作为后续 GraphRAG 索引阶段的执行依据。

● 已完成，新增了两份文件：

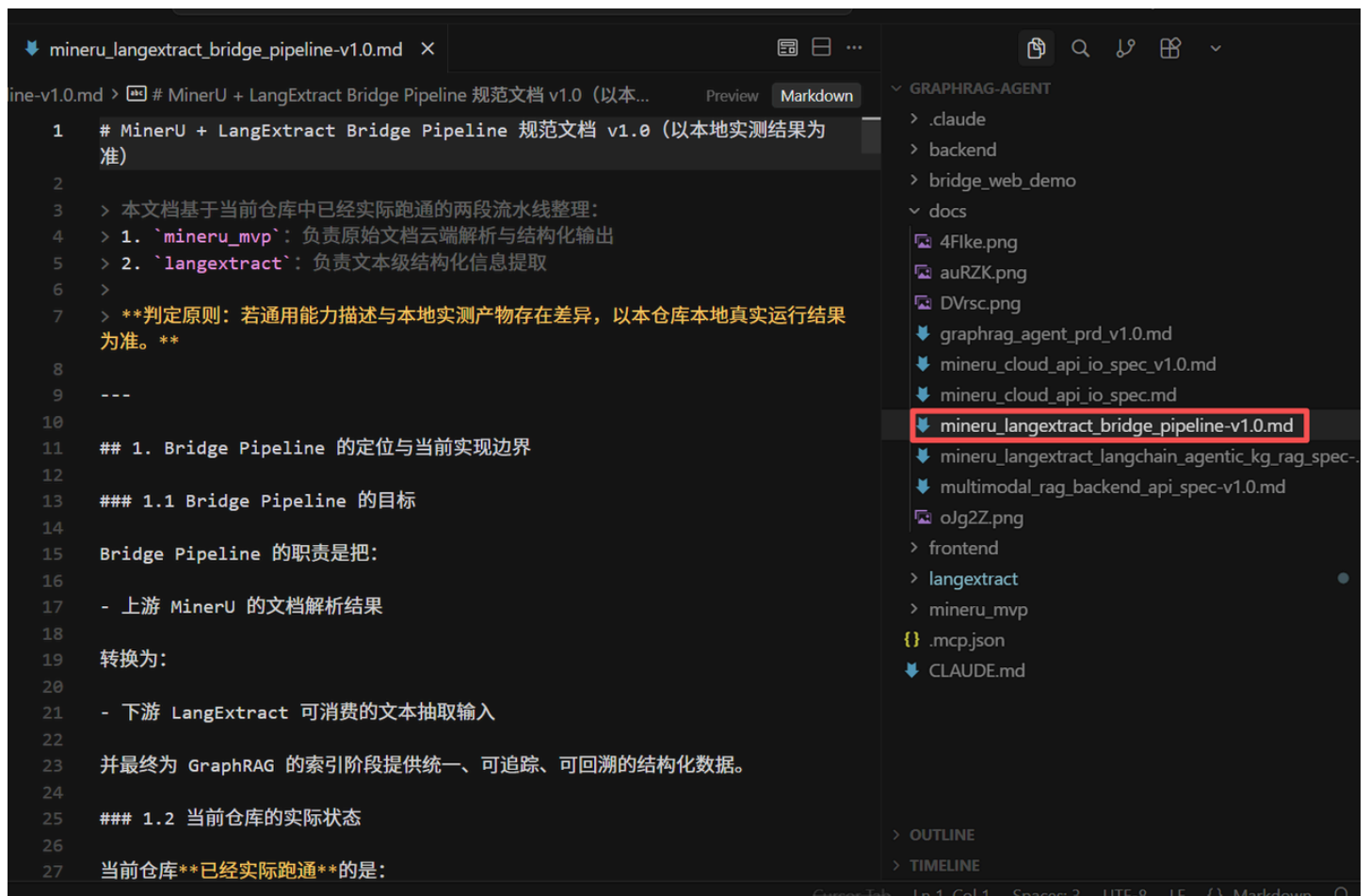
1. Bridge Pipeline 规范文档
 - D:\code\GraphRAG-Agent\docs\mineru_langextract_bridge_pipeline-v1.0.md
2. Bridge 级运行约束
 - D:\code\GraphRAG-Agent\CLAUDE.md

这次处理覆盖了你的 6 点要求

1. 完整 BridgePipeline 执行思路 + 实际测试脚本位置

我已经在文档里明确写了：

- 当前仓库没有独立的 Bridge Pipeline 脚本
- 当前“实际脚本位置”应按两段已验证流程描述：
 - mineru_mvp/run_mvp.py
 - langextract/examples/deepseek_openai_mvp.py



The screenshot shows a code editor with a file named `mineru_langextract_bridge_pipeline-v1.0.md` open. The editor has a dark theme. On the right side, there is a file explorer showing the directory structure of the project, which is named `GRAPHRAG-AGENT`. The file `mineru_langextract_bridge_pipeline-v1.0.md` is highlighted in the file explorer. The main editor area shows the content of the Markdown file, which is a specification document for the Bridge Pipeline. The document is written in Chinese and includes sections for the pipeline's purpose, goals, and current status. The file explorer on the right shows various files and folders, including `.claude`, `backend`, `bridge_web_demo`, `docs`, `4Flke.png`, `auRZK.png`, `DVrsc.png`, `graphrag_agent_prd_v1.0.md`, `mineru_cloud_api_io_spec_v1.0.md`, `mineru_cloud_api_io_spec.md`, `mineru_langextract_langchain_agentic_kg_rag_spec-`, `multimodal_rag_backend_api_spec-v1.0.md`, `oJg2Z.png`, `frontend`, `langextract`, `mineru_mvp`, `.mcp.json`, and `CLAUDE.md`. The `docs` folder is expanded, showing the files within it. The `mineru_langextract_bridge_pipeline-v1.0.md` file is highlighted with a red box in the file explorer.

```
1 # MinerU + LangExtract Bridge Pipeline 规范文档 v1.0 (以本地实测结果为准)
2
3 > 本文档基于当前仓库中已经实际跑通的两段流水线整理：
4 > 1. `mineru_mvp`：负责原始文档云端解析与结构化输出
5 > 2. `langextract`：负责文本级结构化信息提取
6 >
7 > **判定原则：若通用能力描述与本地实测产物存在差异，以本仓库本地真实运行结果为准。**
8
9 ---
10
11 ## 1. Bridge Pipeline 的定位与当前实现边界
12
13 ### 1.1 Bridge Pipeline 的目标
14
15 Bridge Pipeline 的职责是把：
16
17 - 上游 MinerU 的文档解析结果
18
19 转换为：
20
21 - 下游 LangExtract 可消费的文本抽取输入
22
23 并最终为 GraphRAG 的索引阶段提供统一、可追踪、可回溯的结构化数据。
24
25 ### 1.2 当前仓库的实际状态
26
27 当前仓库**已经实际跑通**的是：
```

开展 MinerU 与 LangExtract 两个组件的 MVP 测试，验证二者对接流程的可行性及核心功能的正常运行，为了方便测试我们可以让 Claude Code 开发一个简单的网页。

代码块

- 1 我现在需要实现一个简洁的**单页面HTML可视化工具**，请按照以下需求设计功能：
- 2 1. 支持在线上传 PDF 文档，并自动触发文档解析流程；
- 3 2. 实时展示解析过程中的信息抽取结果；
- 4 3. 基于接口返回的数据，自动构建并渲染可视化知识图谱。
- 5 4. 请严格遵循规范文档 @D:\code\GraphRAG-Agent\docs\mineru_langextract_bridge_pipeline-v1.0.md 完成功能设计与实现。

我现在需要实现一个简洁的**单页面HTML可视化工具**，请按照以下需求设计功能：

1. 支持在线上传 PDF 文档，并自动触发文档解析流程；
2. 实时展示解析过程中的信息抽取结果；
3. 基于接口返回的数据，自动构建并渲染可视化知识图谱。
4. 请严格遵循规范文档 docs\bridge-pipeline-specification-v1.0.md 完成功能设计与实现。

```
isolate-mineru-uv-env × + ▾
打开页面
直接访问：
- http://127.0.0.1:8787
测试流程
1. 选择一个 PDF
2. 点击“开始解析”
3. 观察流程状态变化
4. 查看抽取结果
5. 查看图谱
---
如果你愿意，我下一步可以继续帮你做两件事中的任意一个：
1. 真实上传一个 PDF，替你跑完整链路测试
2. 继续把这版 demo 强化成更稳定版本
   - 增加任务历史
   - 增加失败重试
   - 增加更细的 Bridge 记录展示
   - 增加图谱节点点击详情
* Baked for 1m 21s · 2 shells still running
isolate-mineru-uv-env
>
2 shells · ▶▶ accept edits on
```

← ↻ ⓘ 127.0.0.1:8787

⌵ ☆ ⚙️ | ☆ ⋮

BRIDGE PIPELINE CONSOLE

从 PDF 到结构化抽取，再到可视化知识图谱。

这个单页演示严格遵循 MinerU + LangExtract Bridge Pipeline 规范：上传 PDF，触发解析，展示中间状态、抽取结果，并把最终记录构造成为可浏览的知识图谱视图。

上游解析

MinerU

下游抽取

LangExtract

图谱输出

GraphRAG-ready

上传并启动任务

拖拽 PDF 到这里，或点击选择文件

已选择：大模型微调核心之算力.pdf · 1.82 MB

开始解析

刷新状态

Task: 967541c9e8a0 · 状态: done · 文件: _____.pdf

流程状态

● UPLOADING

文件已经进入系统，等待处理。

● MINERU

MinerU 正在解析原始 PDF，并生成结构化结果。

● BRIDGING

Bridge 正在把 MinerU 的块级结果重组为 LangExtract 可

● LANGEXTRACT

LangExtract 正在进行结构化抽取。

● DONE

所有流程已完成，可浏览抽取数据和图谱。

桥接摘要

任务阶段

done

Bridge 片段

73

运行日志

```
{
  "task_id": "967541c9e8a0",
  "state": "done",
  "stage": "done",
  "message": "Bridge pipeline finished successfully",
  "source_file": "_____.pdf",
  "created_at": "2026-03-26T12:37:58",
```

抽取结果

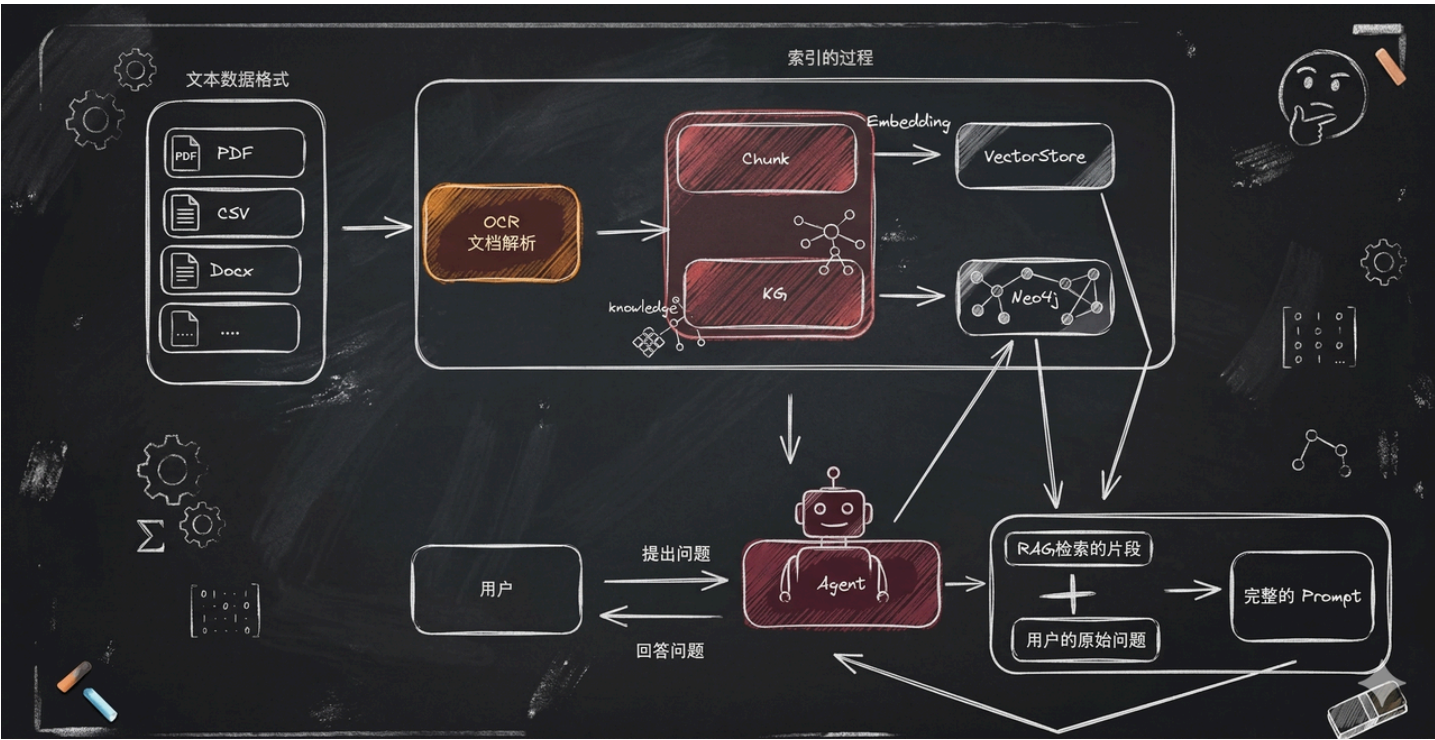
展示 Bridge 标准记录中的 LangExtract 结果，并标记 grounding 状态。

Class	Text	Grounded	Alignment	Attributes
暂无抽取记录。				

页面样式虽然比较简洁，但已经能很好地辅助我们完成流程测试，也验证了当前整体流程可以正常跑通！

6.5 Agent+GraphRAG流程 MVP 测试

接下来我们在前面的流程里面再接入Agent智能体。



接下来要完成的 MVP 流程如下：Agent 接收用户提出的问题后，会先到已构建完成的知识图谱（前序流程已落地）中检索相关结果，再基于检索到的有效信息，整理并生成对用户问题的精准回答。

在我们的整体架构中，还包含将数据存储至向量数据库的分支流程，该部分实现难度较低。本课程将重点为大家演示数据存入知识图谱的核心流程，向量数据库相关的存储流程，大家可以参照课程所教的方法自行完成。

开发 Agent 需要用到 LangChain 开发框架，而 LangChain 的 API 版本迭代较快、兼容性较差。如果直接让 Claude Code 进行开发，很容易出现代码无法运行的问题。因此，最优方案是为 Claude Code 接入 LangChain 的 MCP Server，以此保障代码稳定可执行。

接入 LangChain 的 MCP Server，获取最API规范。

代码块

```
1  ## Claude Code中执行
2  https://docs.Langchain.com/use-these-docs 请安装LangChain 的 MCP 工具，并进行连通性测试！
```

结果如下：

- 已完成安装配置，并做了可达性测试。

结果：

```
- 已创建项目级 MCP 配置：D:\code\GraphRAG-Agent\.mcp.json:1
- 配置内容：
{
  "mcpServers": {
    "langchain-docs": {
      "type": "http",
      "url": "https://docs.langchain.com/mcp"
    }
  }
}
```

连通性测试结果：

```
- https://docs.langchain.com/mcp 可访问
- 返回 HTTP/1.1 200 OK
- Content-Type: application/json
```

说明：

```
- 我按“当前项目本地”做了项目内配置。
- 但 Claude Code 本机的 claude mcp add/list/get 命令在这台 Windows 环境里被 Git Bash 缺失/未配置 卡住了，所以没法在 CLI 内完成二次健康检查。
- 当前阻塞信息是：需要可用的 Git Bash，或设置 CLAUDE_CODE_GIT_BASH_PATH 指向实际 bash.exe。
```

检测是否安装成功：

使用斜杠命令 `/mcp` 测试

```
D:\code\GraphRAG-Agent>claude
Claude Code v2.1.84
Sonnet 4.6 (1M context) · API Usage Billing
D:\code\GraphRAG-Agent

/mcp

Manage MCP servers
2 servers

Project MCPs (D:\code\GraphRAG-Agent\.mcp.json)
> langchain-docs ·  connected
User MCPs (C:\Users\ZhuanZ\claude.json)
pencil ·  connected

https://code.claude.com/docs/en/mcp for help
↑↓ to navigate · Enter to confirm · Esc to cancel
```

在完成 LangChain MCP Server 接入后，由 Claude Code 基于该环境进行整体方案规划与设计。

代码块

- 1 *## Claude Code中执行*
- 2 请你仔细研读 LangChain 官方文档，我需要在 MinerU 与 LangExtract 组件的输出内容基础上，构建 Agentic-RAG 完整流程。请严格参照 D:\code\GraphRAG-Agent\docs\mineru_langextract_bridge_pipeline-v1.0.md 规范，借助 LangChain MCP 工具，规划并输出一套完整、可落地的技术架构方案。

请你仔细研读 LangChain 官方文档，我需要在 MinerU 与 LangExtract 组件的输出内容基础上，构建 Agentic-RAG 完整流程。请严格参照 docs\mineru_langextract_bridge_pipeline-v1.0.md 规范，借助 LangChain MCP 工具，规划并输出一套完整、可落地的技术架构方案。

执行结果如下：

```
---
八、我对当前仓库的最终推荐

推荐你走“先 MVP，后增强”的路线：

- 第一阶段：
MinerU → BridgeChunk → Evidence Index → 基础 RAG + citation
- 第二阶段：
接入 LangExtract 结果 → Fact Index → 实体/关系增强
- 第三阶段：
用 LangGraph 做 agentic retrieval routing，而不是全自治 agent

这条路线最符合你当前仓库的事实边界：
- 还没有独立 bridge 脚本
- MinerU 与 LangExtract 必须维持各自 .venv 边界
- 当前最重要的是把“可追溯证据链”先做实，而不是过早追求复杂 Agent

---
相关官方文档锚点
```

```
- LangGraph Agentic RAG
https://docs.langchain.com/oss/python/langgraph/agentic-rag
- LangChain RAG
https://docs.langchain.com/oss/python/langchain/rag
- LangChain Knowledge Base / Semantic Search
https://docs.langchain.com/oss/python/langchain/knowledge-base
- Retriever Integrations
```

一般方案都是靠谱的，如果不靠谱的情况下，大家可以继续与之对话调整。

接下来让 Claude Code 构建 Agentic-RAG 的 MVP 代码。

代码块

- ```
1 ## Claude Code中执行
2 好的，我已清楚你的方案。请在路径 D:\code\GraphRAG-Agent\bridge_web_demo 下新建一个
 Python 文件，完成 Agentic-RAG 的 MVP 完整流程验证，具体要求如下：
3 1. LangChain 接入大模型时，请使用 DeepSeek 模型，我的 API Key 为：sk-
 1d47dcba4cbb4be4beebf23ef12932f0，请严格遵循 LangChain 官方规范完成接入配置；
4 2. 文档解析及 LangExtract 知识图谱提取的完整规范，可参照 D:\code\GraphRAG-
 Agent\docs\mineru_langextract_bridge_pipeline-v1.0.md 执行；
5 3. 请结合 LangChain 最新版本使用规范，以及此前生成的规划文档，进一步扩展 Agentic-RAG 的
 功能链路，完成全流程运行及连通性测试。
6 注意：无需接入 Embedding 模型进行 Chunk 处理，仅需实现知识图谱（KG）的问答链路即可。
```

好的，我已清楚你的方案。请在路径 integration 目录下新建一个 Python 文件，完成 Agentic-RAG 的 MVP 完整流程验证，具体要求如下：

1. LangChain 接入大模型时，请使用 DeepSeek 模型，我的 API Key 为：sk-1d47dcba4cbb4be4beebf23ef12932f0，请严格遵循 LangChain 官方规范完成接入配置；
2. 文档解析及 LangExtract 知识图谱提取的完整规范，可参照 docs\bridge-pipeline-specification-v1.0.md 执行；
3. 请结合 LangChain 最新版本使用规范，以及此前生成的规划文档，进一步扩展 Agentic-RAG 的功能链路，完成全流程运行及连通性测试。

注意：无需接入 Embedding 模型进行 Chunk 处理，仅需实现知识图谱（KG）的问答链路即可。

接下来，完善 Agentic-RAG 的开发文档规范：

代码块

1    *## Claude Code中执行*

2    现在请结合你实际的执行流程，以及 `agentic_kg_rag_mvp.py` 中 MVP 测试的源码设计，按以下要求完成文档完善与补充：

3    1. 详细梳理 `agentic_kg_rag_mvp.py` 的完整执行思路，并明确该测试脚本的实际存放路径；

4    2. 参考 `D:\code\GraphRAG-Agent\docs\mineru_langextract_bridge_pipeline-v1.0.md`，补充接入 LangChain Agent 后，系统对应的输入输出规范与交互逻辑；

5    3. 依据 `D:\code\GraphRAG-Agent\docs\mineru_cloud_api_io_spec_v1.0.md`，进一步完善与 LangChain 构建 Agentic-RAG 对接的关键参数说明，以及核心技术架构的详细描述；

6    4. 明确定义经过问答流程处理后，最终返回数据的完整格式规范，包括字段含义、数据类型及约束要求。

7    请在 `docs` 文件夹中新建一份专属的规范文档（建议命名为 `xx-v1.0.md`），将上述所有内容整合并规范存储，确保文档逻辑连贯、覆盖全面，作为 Agentic-RAG 流程落地的核心依据。

现在请结合你实际的执行流程，以及 `agentic_rag_kg_mvp.py` 中 MVP 测试的源码设计，按以下要求完成文档完善与补充：

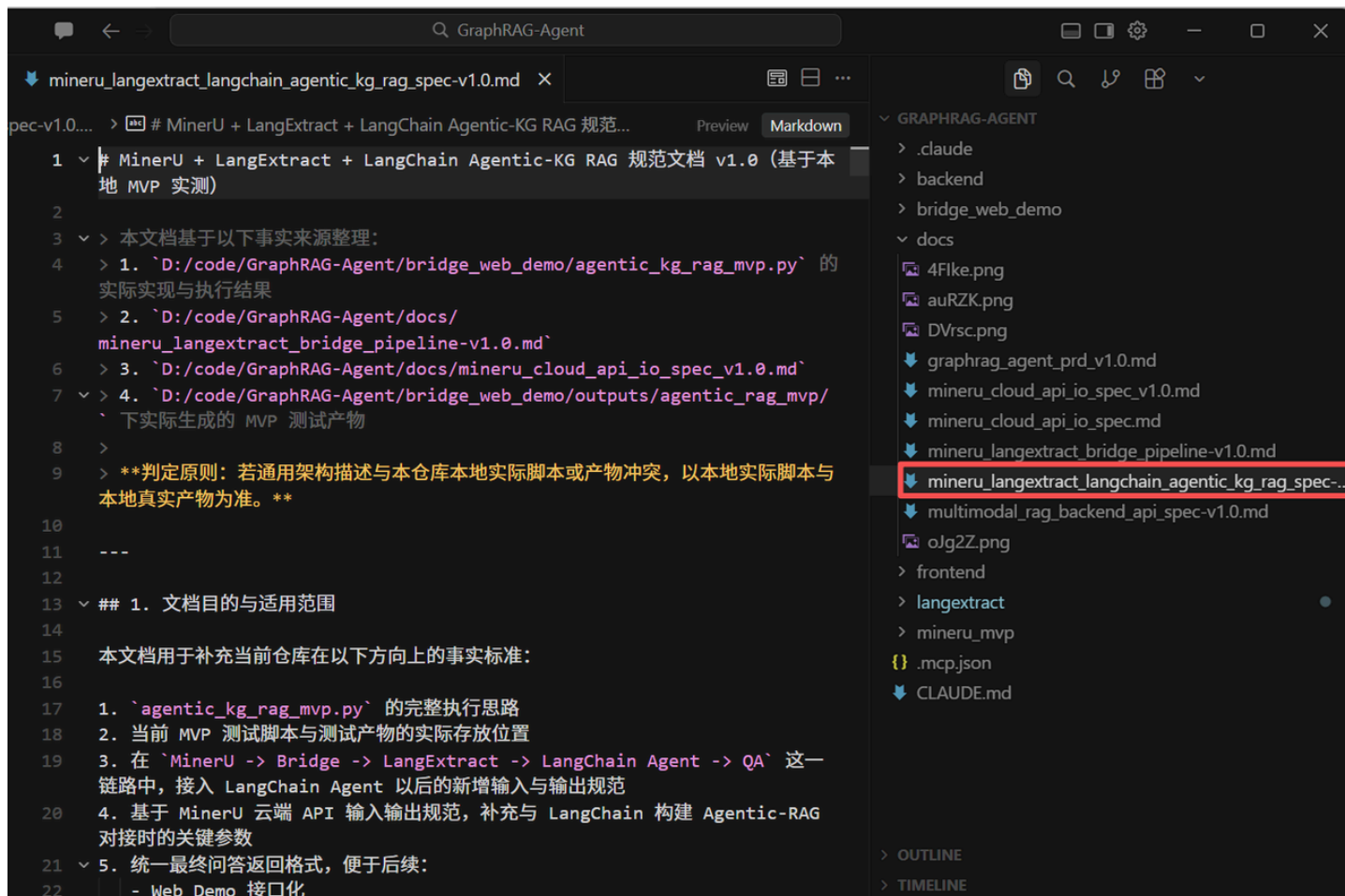
1. 详细梳理 `agentic_kg_rag_mvp.py` 的完整执行思路，并明确该测试脚本的实际存放路径；

2. 参考 `docs\bridge-pipeline-specification-v1.0.md`，补充接入 LangChain Agent 后，系统对应的输入输出规范与交互逻辑；

3. 依据 `docs\LangExtract_规范文档-v1.0.md`，进一步完善与 LangChain 构建 Agentic-RAG 对接的关键参数说明，以及核心技术架构的详细描述；

4. 明确定义经过问答流程处理后，最终返回数据的完整格式规范，包括字段含义、数据类型及约束要求。

请在 `docs` 文件夹中新建一份专属的规范文档（建议命名为 `xx-v1.0.md`），将上述所有内容整合并规范存储，确保文档逻辑连贯、覆盖全面，作为 Agentic-RAG 流程落地的核心依据。

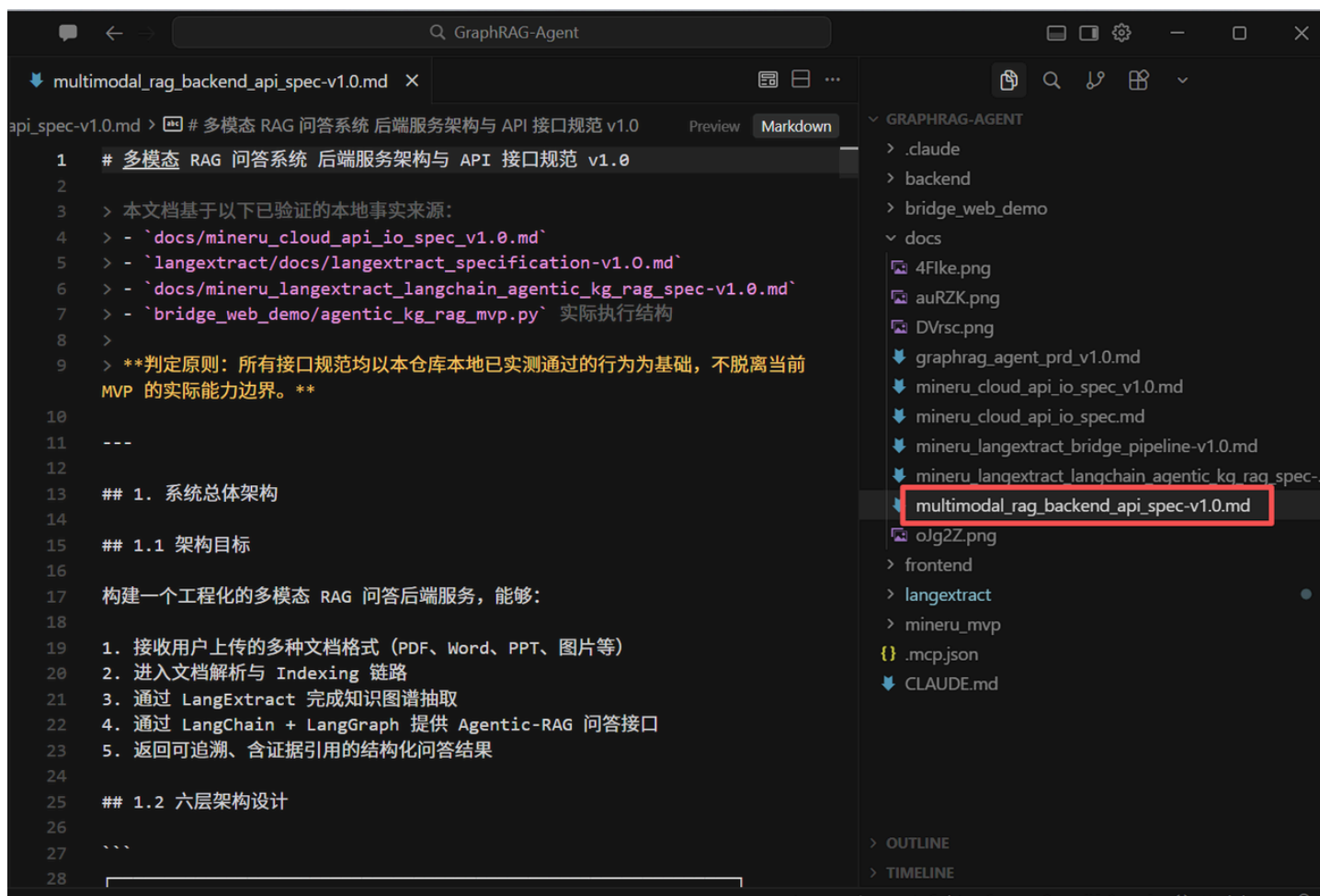


接下来完成后端服务的开发规范编写。

#### 代码块

```
1 ## Claude Code 执行
2 现在我已经构建了完整的后端MVP流程，我需要你帮我开始设计多模态RAG问答系统的后端服务架构以及
 详细的API
3 接口规范，你需要从如下角度考虑：
4 1. docs\MinerU_MVP测试配置指南_v1.0.md 系统的输入需要支持上传多种不同的文件格式到后端的
 服务器，然后进入Indexing 的链路
5 2. docs\LangExtract_规范文档-v1.0.md 包含了核心的知识图谱解析后的数据格式规范
6 3. docs\agentic-rag-kg-specification-v1.0.md 包含了Agentic-RAG 问答的返回数据格式
7 请基于以上信息，我需要构建一个工程化的后端服务接口架构，请详细的帮我完成完整的系统架构设
 计、后端的接口服务的输入和输出规范
8
```

```
agentikg-rag-mvp x + v
你当前有正在需要完成的任务吗？
Read 1 file (ctrl+o to expand)
• 已读完三份核心规范文档，现在有充分信息可以直接输出文档。
• Write(docs\multimodal_rag_backend_api_spec-v1.0.md)
 Wrote 630 lines to docs\multimodal_rag_backend_api_spec-v1.0.md
 1 # 多模态 RAG 问答系统 后端服务架构与 API 接口规范 v1.0
 2
 3 > 本文档基于以下已验证的本地事实来源：
 4 > - `docs/mineru_cloud_api_io_spec_v1.0.md`
 5 > - `langextract/docs/langextract_specification-v1.0.md`
 6 > - `docs/mineru_langextract_langchain_agentikg_rag_spec-v1.0.md`
 7 > - `bridge_web_demo/agentikg_rag_mvp.py` 实际执行结构
 8 >
 9 > **判定原则：所有接口规范均以本仓库本地已实测通过的行为为基础，不脱离当前 MVP 的实际能力边界。**
 ... +620 lines (ctrl+o to expand)
• 好的，已将内容写入 D:/code/GraphRAG-Agent/docs/multimodal_rag_backend_api_spec-v1.0.md.
* Sautéed for 2m 4s
agentikg-rag-mvp
> |
Model: Sonnet 4.6 (1M context) | Ctx: 100.5k | no git | (no git)
plan mode on (shift+tab to cycle)
```



到目前为止，核心流程的所有 MVP 测试工作已全部完成，并且我们已结合测试流程，系统梳理并完善了对应的开发规范文档，为后续正式开发提供了清晰的依据。

## 7、前端开发

接下来和大家分享一个实用经验：尽量不要让 Claude Code 直接一次性生成完整的前后端代码，这样很容易出现问题，而且出错后排查调试难度很大。

更稳妥的方式是前后端分离开发：可以先完成前端代码，再开发后端逻辑；也可以先做好后端，再实现前端界面，两种方式都可行。我个人的习惯是优先开发前端代码。

让 Claude Code 依据此前生成的后端开发规范文档，结合后端接口逻辑与数据交互要求，进行前端页面的设计与开发，确保页面功能与后端完美适配、协同运行

#### 代码块

- 1 *## Claude Code 中执行*
- 2 接下来，请你以资深前端产品设计师的身份，结合当前已有的后端技术架构、可支持输出的数据信息，以及详细的 API 接口规范，为我设计一个可正常运行的 Web 系统原型，并完成全面的规划，具体需包含：系统整体架构、整体 UI 布局风格、响应式适配方案、完整的页面清单，以及详细的交互逻辑等内容。
- 3 帮我规划吧！

agentic-kg-rag-mvp

● 根据后端架构和 API 规范，我直接输出完整的 Web 原型设计方案。

---

多模态 RAG 问答系统 - Web 原型设计方案 v1.0

---

一、整体设计定位

产品类型：企业级智能文档知识问答工作台  
核心价值主张：上传文档 → 自动构建知识图谱 → 基于 KG 证据的可追溯问答  
目标用户：企业知识管理员、研究员、数据分析师  
设计风格：暗色科技风 + 数据可视化优先 + 简洁专业

---

二、UI 设计系统

颜色系统

|      |         |              |
|------|---------|--------------|
| 主背景  | #0F1117 | 深夜黑（主界面底色）   |
| 次级背景 | #1A1D2E | 卡片/面板底色      |
| 边框线  | #2D3147 | 低对比度分割线      |
| 强调蓝  | #4F8EF7 | 主操作按钮、链接、选中态 |
| 强调绿  | #34D399 | 成功/完成状态      |
| 强调橙  | #F59E0B | 进行中/警告状态     |
| 强调红  | #EF4444 | 失败/错误状态      |
| 主文字  | #F1F5F9 | 主要文字         |
| 次级文字 | #94A3B8 | 辅助说明文字       |
| 高亮文字 | #FFFFFF | 标题/重点数据      |

字体系统

英文/数字：Inter (sans-serif)  
中文：PingFang SC / 微软雅黑 fallback  
代码/ID：JetBrains Mono  
标题：22px/Bold  
正文：14px/Regular  
注释：12px/Regular

间距与圆角

|       |                            |
|-------|----------------------------|
| 卡片圆角  | 12px                       |
| 按钮圆角  | 8px                        |
| 输入框圆角 | 8px                        |
| 标签圆角  | 4px                        |
| 基础间距  | 8px 倍数系统（8 / 16 / 24 / 32） |

---

三、整体布局架构

顶部导航栏（固定，64px 高度）  
[Logo] [系统名称] [通知] [设置] [用户头像]

左侧  
导航栏  
(收缩  
展开  
240px)

主内容区  
  
根据当前路由渲染对应页面内容

左侧导航菜单项

 工作台首页

 文档管理

 知识问答

 知识图谱

 分析报告

 系统设置

---

四、页面清单（5 个主页面）

---

Page 1 - 工作台首页 (Dashboard)

路由： /

布局：

|                          |                      |                  |                  |
|--------------------------|----------------------|------------------|------------------|
| 欢迎语 + 快速操作区（上传文档 / 开始问答） |                      |                  |                  |
| 已处理<br>文档数<br>28         | KG 实体<br>总数<br>1,420 | 今日问答<br>次数<br>86 | 成功率<br><br>94.2% |

|                          |  |
|--------------------------|--|
| 最近文档处理列表（状态 / 时间 / 快速入口） |  |
| 最近问答历史（最近 5 条，含来源文档标签）   |  |

- 交互逻辑：
- 顶部快速上传按钮 → 打开上传 Modal
  - 开始问答按钮 → 跳转至问答页
  - 点击任意文档行 → 跳转至文档详情
  - 点击历史问答 → 恢复该问答会话

Page 2 – 文档管理 (Documents)

路由：/documents

布局：

标题 + [上传文档] 按钮

[搜索框]

筛选栏：全部 | 处理中 | 已完成 | 失败

文档列表（表格视图）

| 文件名    | 格式  | 状态  | 实体数 | 时间    | 操作    |
|--------|-----|-----|-----|-------|-------|
| xx.pdf | PDF | ✅完成 | 66  | 03-26 | 查看/问答 |
| yy.pdf | PDF | 🕒中  | -   | 03-26 | 查看进度  |

子组件 – 上传 Modal：

|                                                                        |        |
|------------------------------------------------------------------------|--------|
| 上传文档                                                                   | ×      |
| 拖放文件到此处<br>或点击选择文件<br>支持 PDF / DOCX / PPT                              |        |
| 高级选项（可折叠）<br>语言：[中文 ▼]<br>启用表格识别：[ON]<br>启用公式识别：[OFF]<br>解析页范围：[_____] |        |
| [取消]                                                                   | [开始上传] |

子页面 – 文档详情 /documents/{task\_id}：

|                        |                                             |
|------------------------|---------------------------------------------|
| ← 返回                   | 文档名称                                        |
| 处理<br>进度条              | 基本信息<br>文件名 / 状态<br>页数 / 实体数<br>batch_id 追踪 |
| Stage 1<br>MinerU      | ✅                                           |
| Stage 2<br>Bridge      | ✅ 抽取结果摘要                                    |
| Stage 3<br>LangExtract | 🕒 实体分类统计                                    |
| Stage 4<br>KG 构建       | 按类型的饼图                                      |
|                        | [查看完整图谱]                                    |
|                        | [开始问答]                                      |

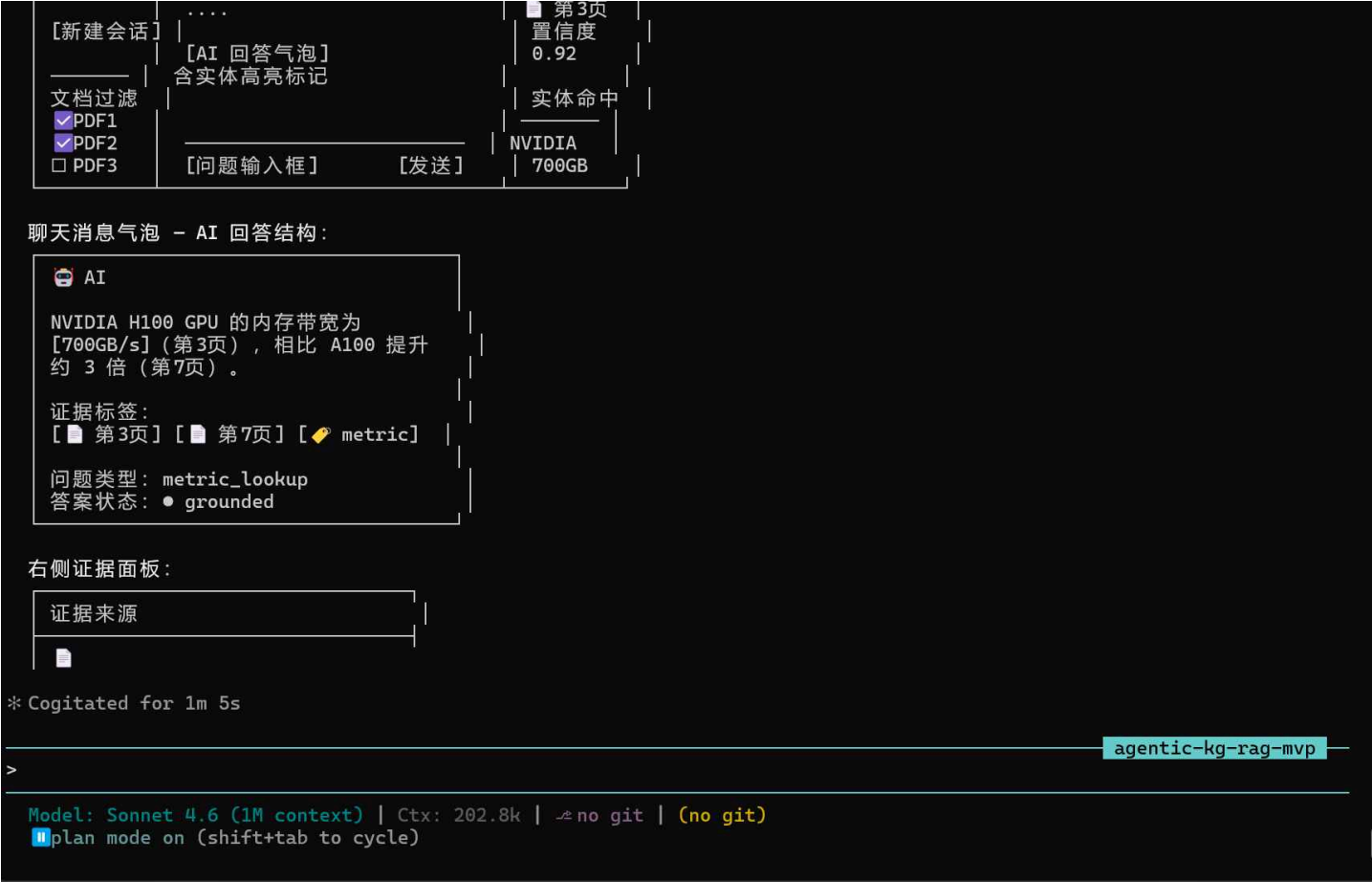
- 交互逻辑：
- 拖拽区支持拖放上传；点击触发文件选择器
  - 上传完成后自动关闭 Modal，文档列表刷新并出现新行，状态为"处理中"
  - 每 3 秒轮询 GET /api/v1/documents/{task\_id}/status
  - Stage 进度条实时更新（state 字段驱动）
  - "查看完整图谱"→ 跳转 Page 3 知识图谱页，携带 task\_id
  - "开始问答"→ 跳转 Page 2 知识问答页，携带 task\_id 作为默认数据源

Page 3 – 知识问答 (Chat)

路由：/chat、/chat/{session\_id}

布局（三栏设计）：

|                     |           |                     |
|---------------------|-----------|---------------------|
| 左侧会话<br>列表<br>280px | 主聊天区      | 右侧证据<br>面板<br>320px |
|                     | [AI 回答气泡] |                     |
| 历史会话1               | ....      | 证据来源                |
| 历史会话2               | [用户问题气泡]  |                     |



如果有问题，可以继续交互，确保没问题为止。

接下来基于产品原型生成PRD文档。

```
代码块

1 ## Claude Code 中执行
2 基于你上述设计的产品原型，
3 我需要你结合实际的后端系统架构规范文档:docs\multimodal-rag-backend-api-specification-v1.0.md 给我撰写个完整的、结构化及标准的产品需求文档，主要从以下几方面输出：
4 1. 项目的核心目标及核心场景，以及处理的业务痛点、面向的用户群体等
5 2. 产品流程，要包含从入口到最终目标的每一步执行流程,需要进行详细描述
6 3. 关键页面/模块的清单和设计说明
7 4. 关键节点和核心功能的交互逻辑
8 5. 整体的UI布局、配色要求、响应式和组件样式等
9 请帮我直接生成PRD文档，以xxx_PRD_1.0.md的文件形式存储
```

PRD文档生成

```
● PRD 文档已完整写入：

文件路径：D:/code/GraphRAG-Agent/docs/graphrag_agent_prd_v1.0.md

文档涵盖了你要求的全部 5 个维度：

1. 项目核心目标与业务场景
- 5 个核心量化目标（问答准确率 ≥85%、处理成功率 ≥95%、响应时间 ≤8s 等）
- 4 个核心使用场景（研究报告、多文档对比、合规审查、图谱探索）
- 5 大业务痛点与解法对照表
- 5 类目标用户角色与诉求

2. 产品流程
- 主流程完整时序：从入口上传 → MinerU 解析（5 个 Stage）→ KG 构建 → 问答 → 证据查看
- 3 条次要流程：图谱探索、多轮对话、失败恢复
- 文档状态机全链路（uploading → pending → processing → completed / failed）

3. 页面/模块清单与设计说明
- 8 个页面的完整说明（P-01 到 P-05-A）
- 每个页面含：模块清单、字段规范、API 依赖
- 全局组件规范（侧边栏、Toast、空状态）
```

你是否还记得上一章我们在 Claude Code 中接入的 Pencil MCP 工具？它是一款专业的页面设计工具，本次我们就直接使用 Pencil 来完成页面设计。

```
代码块

1 ## Claude Code 中执行
2 基于 docs/multimodal-rag_PRD_1.0.md文档，用 pencil 工具生成产品原型图，原型图里涉及到的文字用中文显示。
3
```

基于 docs/multimodal-rag\_PRD\_1.0.md文档，用 pencil 工具生成产品原型图，原型图里涉及到的文字用中文显示。

```
● 原型图已全部生成完成！让我截图做最终确认。

● pencil - get_screenshot (MCP)(filePath: "pencil-new.pen", nodeId: "auRZK")
 | Screenshot image generated successfully for node: auRZK. Analyze the image that it satisfies the requirements
 | and doesn't contain any visual errors, glitches or misalignment.
 | [Image]

● pencil - get_screenshot (MCP)(filePath: "pencil-new.pen", nodeId: "DVrsc")
 | Screenshot image generated successfully for node: DVrsc. Analyze the image that it satisfies the requirements
 | and doesn't contain any visual errors, glitches or misalignment.
 | [Image]

● pencil - get_screenshot (MCP)(filePath: "pencil-new.pen", nodeId: "4FIke")
 | Screenshot image generated successfully for node: 4FIke. Analyze the image that it satisfies the requirements
 | and doesn't contain any visual errors, glitches or misalignment.
 | [Image]

● pencil - get_screenshot (MCP)(filePath: "pencil-new.pen", nodeId: "oJg2Z")
 | Screenshot image generated successfully for node: oJg2Z. Analyze the image that it satisfies the requirements
 | and doesn't contain any visual errors, glitches or misalignment.
 | [Image]

● 产品原型图已全部生成完成！共创建了 4个核心页面，所有文字均使用中文：

P01 - 工作台首页
- 深色侧边导航栏（工作台/文档管理/知识问答/知识图谱/系统设置）
- 4个数据统计卡片：已处理文档(128)、KG实体总数(45,231)、今日问答(342)、成功率(92.5%)
- 最近文档列表（含状态标签：已完成/处理中）
- 最近问答历史（含有据可查/部分支撑 状态标记）

P02 - 文档管理页
- 搜索框 + 上传文档按钮
- 全部/处理中/已完成/失败 Tab筛选
- 文档列表表格：文件名、格式、状态、实体数、处理时间、操作按钮
- 不同状态对应不同操作（查看详情/查看图谱/查看进度/重新处理）

P03 - 知识问答页（三栏布局）
- 左侧：新建会话、数据源多选、历史会话列表
- 中间：用户消息气泡（蓝色右对齐）+ AI回答气泡（含"有据可查"状态徽标）+ 底部输入框
- 右侧证据面板：原文引用、页码、完全匹配/部分匹配状态、查看原文链接

P04 - 知识图谱页
- 深色背景力导向图：英伟达/AMD/英特尔/华为昇腾等节点（按类型差异化着色）
- 左上角：实体搜索框 + 图例（机构/产品/指标/时间）
- 右侧控制按钮（缩放/全屏）
- 右侧节点详情抽屉：实体名称、类型、属性列表、关联关系、"以此节点发起问答"按钮

* Cooked for 12m 50s

>
```

生成4张原型图

🖥️ > 此电脑 > 本地磁盘 (D:) > code > GraphRAG-Agent > docs

📁 📄 🔄 🗑️ ⬆️ 排序 ▾ ≡ 查看 ▾ ⋮

| 名称                                                       | 修改日期            | 类型            | 大小 |
|----------------------------------------------------------|-----------------|---------------|----|
| 4Flke.png                                                | 2026/3/26 20:40 | PNG 图片文件      |    |
| auRZK.png                                                | 2026/3/26 20:40 | PNG 图片文件      |    |
| DVrsc.png                                                | 2026/3/26 20:40 | PNG 图片文件      |    |
| graphrag_agent_prd_v1.0.md                               | 2026/3/26 15:20 | Markdown File |    |
| mineru_cloud_api_io_spec.md                              | 2026/3/26 0:14  | Markdown File |    |
| mineru_cloud_api_io_spec_v1.0.md                         | 2026/3/26 10:09 | Markdown File |    |
| mineru_langextract_bridge_pipeline-v1.0.md               | 2026/3/26 11:54 | Markdown File |    |
| mineru_langextract_langchain_agentic_kg_rag_spec-v1.0.md | 2026/3/26 14:18 | Markdown File |    |
| multimodal_rag_backend_api_spec-v1.0.md                  | 2026/3/26 14:48 | Markdown File |    |
| oJg2Z.png                                                | 2026/3/26 20:40 | PNG 图片文件      |    |

样式如下：

|    |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |
|----|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 图1 | <div><div><div>GraphRAG</div><div>工作台</div><div>文档管理</div><div>知识问答</div><div>知识图谱</div><div>系统设置</div></div><div><div>工作台概览</div><div><div>已处理文档<br/>128</div><div>KG实体总数<br/>45,231</div><div>今日问答次数<br/>342</div><div>问答成功率<br/>92.5%</div></div><div><div>最近文档</div><div><div>2024年AI芯片市场分析报告.pdf<br/>2026-03-25 · 实体: 234</div><div>竞品对比分析-腾讯华为阿里.docx<br/>2026-03-24 · 实体: 156</div><div>合同模板-服务采购协议2025.pdf<br/>2026-03-23 · 实体: 89</div></div><div><div>最近问答</div><div><div>2024年AI芯片市场规模是多少?<br/>AI芯片报告 ✓ 有据可查 10分钟前</div><div>对比三家公司的云计算业务营收增长策略<br/>竞品分析 ⚠️ 部分支撑 1小时前</div><div>合同中违约金的方式是什么?<br/>服务协议 ✓ 有据可查 昨天</div></div></div></div></div></div> |
| 图2 |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |

GraphRAG

工作台

文档管理

知识问答

知识图谱

系统设置

文档管理

全部处理中已完成失败

| 文件名                 | 格式   | 状态  | 实体数 | 处理时间             | 操作                  |
|---------------------|------|-----|-----|------------------|---------------------|
| 2024年AI芯片市场分析报告.pdf | PDF  | 已完成 | 234 | 2026-03-25 14:32 | <div>查看详情查看图谱</div> |
| 竞品对比分析-腾讯华为阿里.docx  | DOCX | 处理中 | --  | 2026-03-24 09:15 | <div>查看进度</div>     |
| 合同模板-服务采购协议2025.pdf | PDF  | 已完成 | 89  | 2026-03-23 16:48 | <div>查看详情查看图谱</div> |
| 产品白皮书-云计算平台架构.pptx  | PPTX | 失败  | --  | 2026-03-22 11:20 | <div>查看原因重新处理</div> |

图3

GraphRAG

工作台

文档管理

知识问答

知识图谱

系统设置

知识图谱

搜索实体节点...

图例

机构 / 公司

产品

指标

时间

AMD

英特尔

2024Q3

华为昇腾

GPU市场

英伟达

679亿USD

节点详情

英伟达 (NVIDIA)

实体类型: ORGANIZATION

属性

市场份额65%  
主营产品数据中心GPU  
出现次数23次

关联关系 (4)

主导GPU市场

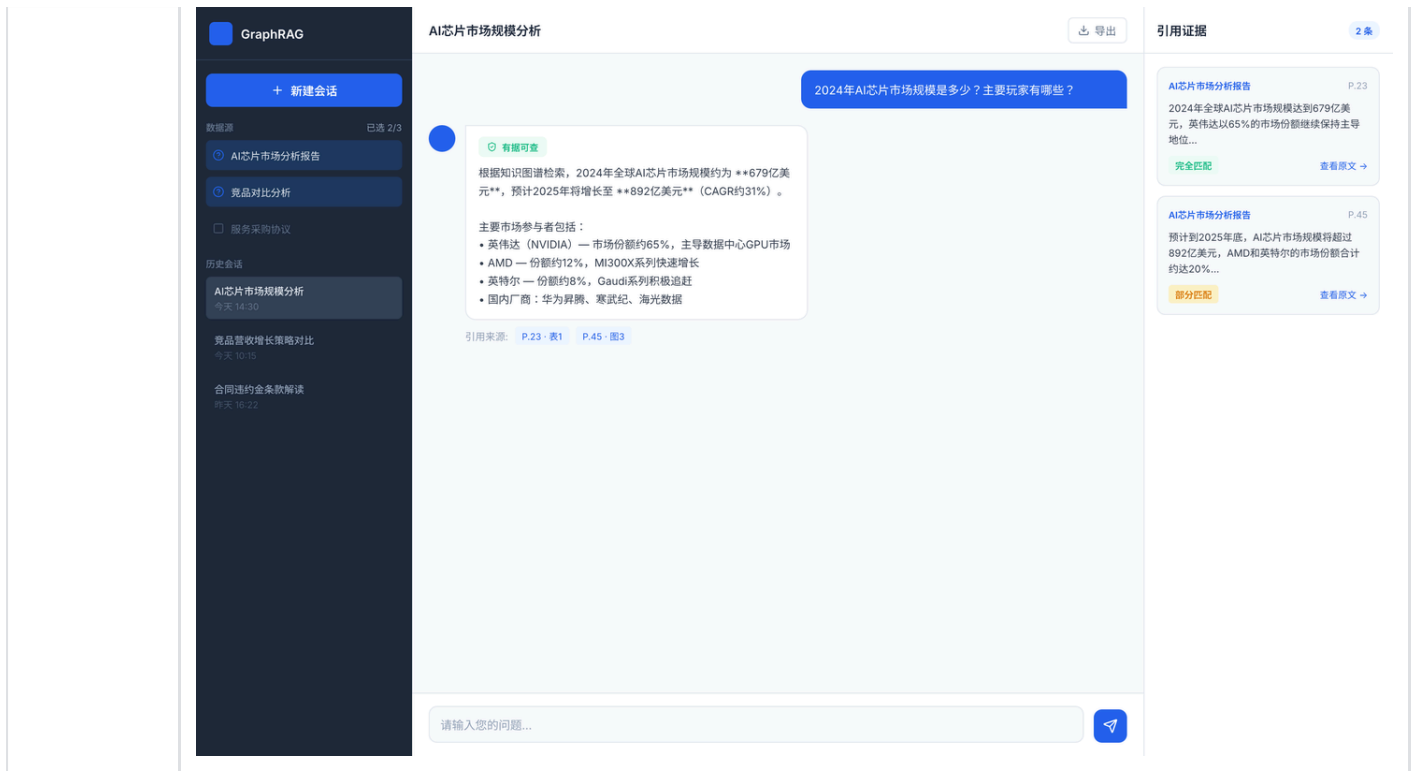
竞争AMD

产品H100 GPU

以此节点发起问答

查看来源文档

图4



如果觉得不满意的地方，可以继续交互。

在开发项目代码前，编写开发规范。

代码块

- 1
- ## Claude Code中执行
- 2
- 请你在当前的根目录下 写一个CLAUDE.md 文件，写明如下规范：注意，一定要简洁：
- 3
1. 项目结构：前端的所有代码结构需要放在一个frontend文件夹下，后端的所有代码结构， 请放在 backend文件夹下
- 4
2. 后端的所有外部配置文件比如api key等，全部在.env 文件中管理，同时不要将.env 文件信息提交到git中，务必在
- 5
- gitignore中忽略
- 6
3. 后端的服务务必使用uv创建独立的虚拟环境

Cursor 中使用 rules

项目开发规范：1. 前端所有代码统一放在 frontend/ 文件夹下 2. 后端所有代码统一放在 backend/ 文件夹下 3. 后端敏感配置（API Key等）统一在 .env 文件管理，.env 必须加入 .gitignore 禁止提交 Git 4. 后端服务必须使用 uv 创建独立虚拟环境

```
agentic-kg-rag-mvp x + v
56 +Ensure the following are ignored:
57
85 -The repository does not yet have a standalone Bridge Pipeline script.
86 -The current verified bridge flow is formed by these two entrypoints:
87 -
88 -- `D:/code/GraphRAG-Agent/mineru_mvp/run_mvp.py`
89 -- `D:/code/GraphRAG-Agent/langextract/examples/deepseek_openai_mvp.py`
90 -
91 -Any execution of the current bridge flow must respect the environment boundary rules above.
58 +```
59 +.env
60 +**/.env
61 +**/.venv/
62 +```

● 已更新根目录 CLAUDE.md, 简洁覆盖了你要求的三条规范:

1. 项目结构: 前端代码放 frontend/, 后端代码放 backend/, 保留现有 mineru_mvp/ / langextract/ / bridge_web_demo/ 原有规则不变。
2. 后端配置管理: 所有 API Key 等外部配置统一在 backend/.env 管理, 不提交 git, .gitignore 中明确忽略 .env 和 .venv/.
3. 后端运行时: 必须用 uv 创建独立虚拟环境 backend/.venv, 禁止使用全局 Python 或其他组件的 .venv。

* Cooked for 56s

agentic-kg-rag-mvp
>
Model: Sonnet 4.6 (1M context) | Ctx: 241.1k | no git | (no git)
plan mode on (shift+tab to cycle)
```

接下来将借助 Claude Code 进行前端代码的生成，这部分需要较长时间的编码实现。

```
代码块
1 ## Claude Code
2 很好，根据刚刚生成的产品原型图，基于@D:/code/GraphRAG-Agent/docs/graphrag_agent_prd_v1.0.md文档
3 使用ReAct前端框架，生成前端代码，代码写到当前目录的frontend文件夹下。
```

很好，根据刚刚生成的产品原型图，基于 doc\multimodal-rag\_PRD\_1.0.md文档  
使用ReAct前端框架，生成前端代码，代码写到当前目录的frontend文件夹下。

● 前端代码全部生成完成！以下是完整总结：

---

完成情况

构建状态： ✓ TypeScript 类型检查通过 · ✓ Vite build 成功 (6.11s, 1649 modules)

生成的文件结构 (35个源文件)

```
frontend/src/
├── main.tsx / App.tsx # 入口 + 路由
├── types/ # TS类型 (与后端严格对应)
│ ├── document.ts # TaskState枚举、TaskStatus
│ ├── graph.ts # 图谱节点/边 + Cytoscape格式
│ ├── agent.ts # 问答请求/响应 + 会话消息
│ └── api.ts # ApiError类
├── api/ # Axios 层
│ ├── client.ts # 实例 + 统一错误拦截
│ ├── documents.ts / graph.ts / agent.ts
│ └── keys.ts # QueryKey工厂
├── store/ # Zustand
│ ├── chatStore.ts # 消息队列 + 会话状态
│ ├── graphStore.ts # 节点选中 + 视图过滤
│ └── uiStore.ts # Modal开关 + Toast队列
├── hooks/ # TanStack Query封装
│ ├── useDocuments.ts # 列表 (处理中自动5s轮询)
│ ├── useDocumentStatus.ts # 单文档3s轮询
│ ├── useUploadDocument.ts # 上传mutation
│ ├── useAgentQuery.ts # 问答 (自动注入loading气泡)
│ └── useSessions.ts # 会话列表 + 删除
├── components/
│ ├── layout/ Sidebar + AppShell
│ ├── ui/ StatusBadge + StatCard + FileIcon + ToastContainer
│ └── modals/ UploadModal (拖拽上传 + 配置选项)
├── pages/
│ ├── Dashboard/ 工作台 (统计卡片 + 最近文档 + 最近问答)
│ ├── Documents/ 文档管理 (Tab筛选 + 表格 + 状态轮询)
│ ├── Chat/ 知识问答 (三栏布局 + 消息气泡 + 证据面板)
│ └── Graph/ 知识图谱 (Cytoscape.js + fcose布局 + 节点详情抽屉)
```

启动方式

```
cd D:/code/GraphRAG-Agent/frontend
npm run dev
访问 http://localhost:5173
后端需同时运行于 http://127.0.0.1:8000 (已通过 Vite 代理自动转发 /api 请求)。
```

\* Cooked for 53m 53s

>

```
Model: Sonnet 4.6 (1M context) | Ctx: 205.0k | ↗ no git | (no git)
▶▶ accept edits on (shift+tab to cycle)
```

前端页面开发完成！

效果如下：



## 8、后端开发

根据开发规范生成后端代码：

代码块

- 1 `## 在 Claude Code 中执行`
- 2 `D:\code\GraphRAG-Agent\docs\multimodal_rag_backend_api_spec-v1.0.md`
- 3 请严格按照文档规范，构建GraphRAG的后端项目架构并自动进行项目启动和接口测试。注意:这是一个全新的项目，你可
- 4 以参考规范文档中提及的测试文件中的功能执行流程，但一定要在新项目中生成独立的文件服务，不要直接引用

请严格按照文档规范 docs\multimodal-rag-backend-api-specification-v1.0.md，构建GraphRAG的后端项目架构并自动进行项目启动和接口测试。注意:这是一个全新的项目，你可  
以参考规范文档中提及的测试文件中的功能执行流程，但一定要在新项目中生成独立的文件服务，不要直接引用

agentic-kg-rag-mvpClaude Code

● 服务日志已确认全部核心接口均正常运行。

服务器日志摘要显示所有接口都被成功调用并正确响应：

| 接口                                | HTTP 状态       | 说明                 |
|-----------------------------------|---------------|--------------------|
| GET /health                       | 200 OK        | 服务健康检查通过           |
| GET /api/v1/documents             | 200 OK        | 文档列表正常             |
| POST /api/v1/retrieve/text        | 200 OK        | 文本检索正常             |
| GET /api/v1/agent/sessions        | 200 OK        | 会话列表正常             |
| POST /api/v1/graph/subgraph       | 404 Not Found | 预期行为（无完成任务时返回 404） |
| POST /api/v1/documents/upload     | 202 Accepted  | 文档上传成功，流水线启动       |
| GET /api/v1/documents/{id}/status | 200 OK        | 任务状态查询正常           |
| GET /api/v1/graph/node/{id}       | 200 OK        | 图谱节点查询正常           |
| GET /api/v1/graph/neighbors/{id}  | 200 OK        | 邻居节点查询正常           |
| POST /api/v1/agent/query          | 200 OK        | Agent 问答接口完整返回     |

后端服务已完整构建并验证通过，所有 20 个单元测试通过，所有 API 接口连通性测试正常，FastAPI 服务在 http://127.0.0.1:8000 正常运行。

agentic-kg-rag-mvp>

Model: Sonnet 4.6 (1M context) | Ctx: 273.1k | no git | (no git)

编写前后端的CLAUDE.md，方便后面对项目的管理。

代码块

1 ## 在 Claude Code 中执行

2 请帮我在backend文件夹下面写一个启动后端项目的CLAUDE.md，表明在哪个路径下、使用什么命令。

● Write(D:/code/GraphRAG-Agent/backend/CLAUDE.md)

Wrote 71 lines to ..\backend\CLAUDE.md

1 # CLAUDE.md - Backend

2

3 ## 路径

4

5 所有后端操作均在 `D:/code/GraphRAG-Agent/backend/` 目录下执行。

6

7 ---

8

9 ## 环境准备（首次运行）

... +61 lines (ctrl+o to expand)

● 已在 backend/CLAUDE.md 写好，内容包括：

- 路径：明确所有操作在 D:/code/GraphRAG-Agent/backend/ 下

- 首次环境准备：uv venv + uv pip install -e .

- 配置 .env: 从 .env.example 复制后填入三个关键 API Key

- 启动命令：uv run python main.py（服务监听 http://127.0.0.1:8000）

- 验证方式：health check 接口和 Swagger 文档地址

- 注意事项：venv 隔离、.env 不提交 git 等

\* Churned for 2m 36s

>

Model: Sonnet 4.6 (1M context) | Ctx: 210.1k | no git | (no git)

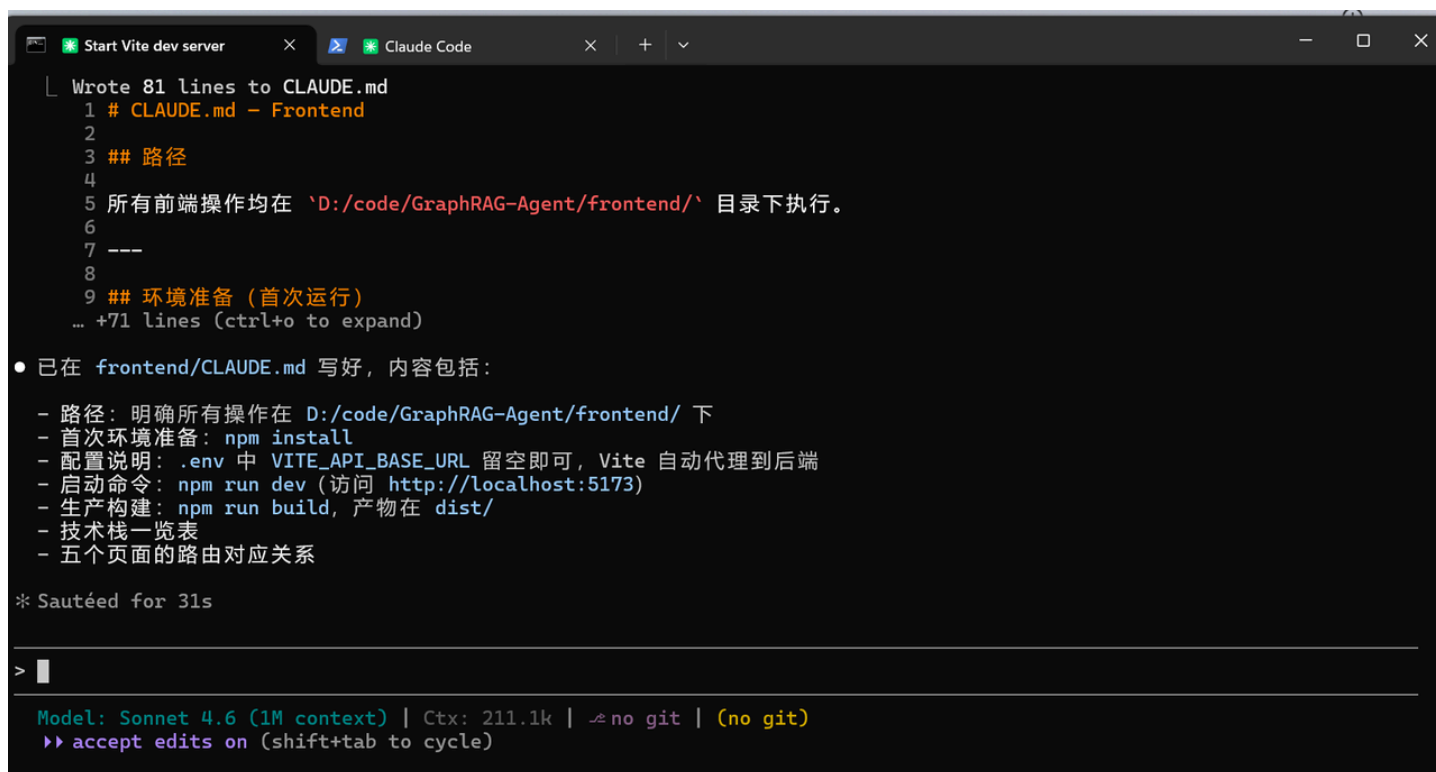
▶ accept edits on (shift+tab to cycle)

#### 代码块

- 1 `## 在 Claude Code 中执行`
- 2 请帮我在frontend文件夹下面写一个启动前端项目的CLAUDE.md，表明在哪个路径下、使用什么命令。

请你结合Cursor的rules规范，帮我写一个rules，里面表明 前端代码在哪个路径下、使用什么命令启动。

后端代码在哪个路径下，使用什么命令启动。



```
Wrote 81 lines to CLAUDE.md
1 # CLAUDE.md - Frontend
2
3 ## 路径
4
5 所有前端操作均在 `D:/code/GraphRAG-Agent/frontend/` 目录下执行。
6
7 ---
8
9 ## 环境准备 (首次运行)
... +71 lines (ctrl+o to expand)

• 已在 frontend/CLAUDE.md 写好，内容包括：
- 路径：明确所有操作在 D:/code/GraphRAG-Agent/frontend/ 下
- 首次环境准备：npm install
- 配置说明：.env 中 VITE_API_BASE_URL 留空即可，Vite 自动代理到后端
- 启动命令：npm run dev (访问 http://localhost:5173)
- 生产构建：npm run build, 产物在 dist/
- 技术栈一览表
- 五个页面的路由对应关系

* Sautéed for 31s

> |

Model: Sonnet 4.6 (1M context) | Ctx: 211.1k | no git | (no git)
▶▶ accept edits on (shift+tab to cycle)
```

## 9、前后端联调

接下来我们进行后端代码的开发。这里分享一个使用 AI 编程的核心经验：由于我们采用 AI 辅助开发，前后端功能很容易出现接口、逻辑不匹配的情况。因此在生成代码时，一定要以其中一端为主导，暂时无法匹配的功能先预留空位，不要强行同步开发。如果直接让 AI 同时生成前后端代码，一旦发现功能不匹配，它会自动补全逻辑，极易产生错误，而且后期排查难度非常大。

这个经验非常宝贵！

#### 代码块

- 1 `## Claude Code 中执行`
- 2 我现在需要你仔细的阅读后端服务接口的设计，参考：D:\code\GraphRAG-Agent\docs\multimodal\_rag\_backend\_api\_spec-v1.0.md
- 3 完成前后端接口的对接，你需要做的事情是：
- 4 1. 将前端源码中所有功能组件的mock API全部替换为真实的后端API接口服务。

- 5     2. 在完成对接后，需要逐个进行前后端接口的集成测试，注意:如果遇到前端需要的接口在后端并没有开发实现，千万不要去尝试自主的开发后端新的接口，仅需匹配当前版本的后端接口，要以目前的后端接口设计为准!
- 6             同时，在前端的功能页面上给我打上未开发的标识!!
- 7

我现在需要你仔细的阅读后端服务接口的设计，参考:docs\multimodal-rag-backend-api-specification-v1.0.md

完成前后端接口的对接，你需要做的事情是：

1. 将前端源码中所有功能组件的mock API全部替换为真实的后端API接口服务。
2. 在完成对接后，需要逐个进行前后端接口的集成测试，

注意:如果遇到前端需要的接口在后端并没有开发实现，千万不要去尝试自主的开发后端新的接口，仅需匹配当前版本的后端接口，要以目前的后端接口设计为准!

同时，在前端的功能页面上给我打上未开发的标识!!

- 构建成功。
- 所有接口对接和修复已完成，构建成功（✓ 1650 modules）。以下是完整总结：

---  
修复内容汇总

已修复的接口问题

| 文件                     | 问题                                                         | 修复方式                               |
|------------------------|------------------------------------------------------------|------------------------------------|
| api/client.ts          | 错误格式解析错误（data.error 是对象非字符串）                               | 正确解析 {error: {code, message}} 嵌套结构 |
| api/documents.ts       | 调用不存在的 GET /api/v1/documents 列表接口                          | 移除，改为本地 store 管理                   |
| api/documents.ts       | 上传参数缺少 is_ocr、page_ranges、extraction_profile、model_version | 补全所有上传参数                           |
| api/agent.ts           | 调用不存在的会话列表/历史/删除接口                                         | 全部移除，仅保留 query 和 chat              |
| api/graph.ts           | getNeighbors 传了后端不支持的 task_id 参数                           | 改为只传 depth 和 relation_type         |
| types/agent.ts         | AgentQueryRequest 包含后端不接受的 task_ids                        | 删除，响应字段改为可选                        |
| hooks/useAgentQuery.ts | 传了后端不支持的 task_ids；首次/续轮对话路由错误                              | 去除 task_ids，自动区分 query/chat        |
| store/chatStore.ts     | 包含无效的 selectedTaskIds 数据源选择逻辑                              | 精简，移除无效状态                          |
| pages/Documents        | 假删除（用 GET 接口伪造）、无实体数字段                                     | 改为真实状态轮询 + 本地记录移除                  |
| pages/Graph            | 错误调用 subgraph 加载整图                                         | 改用正确的 GET /documents/{id}/graph    |

新增机制

- store/documentStore.ts: 基于 zustand/persist 的 localStorage 持久化文档列表，解决无列表接口问题
- DocumentRow 组件：每行独立轮询自身状态，自动更新 store

标注为 "未开发" 的功能

| 功能           | 位置              | 原因                             |
|--------------|-----------------|--------------------------------|
| 历史会话列表       | Chat 页左侧边栏      | 需后端 GET /api/v1/agent/sessions |
| 今日问答次数 / 成功率 | Dashboard 统计卡片  | 依赖会话接口                         |
| 最近问答记录       | Dashboard 右下卡片  | 依赖会话接口                         |
| 重新处理按钮       | Documents Tab 栏 | 需后端删除/重处理接口                    |

\* Cogitated for 28m 5s

启动服务

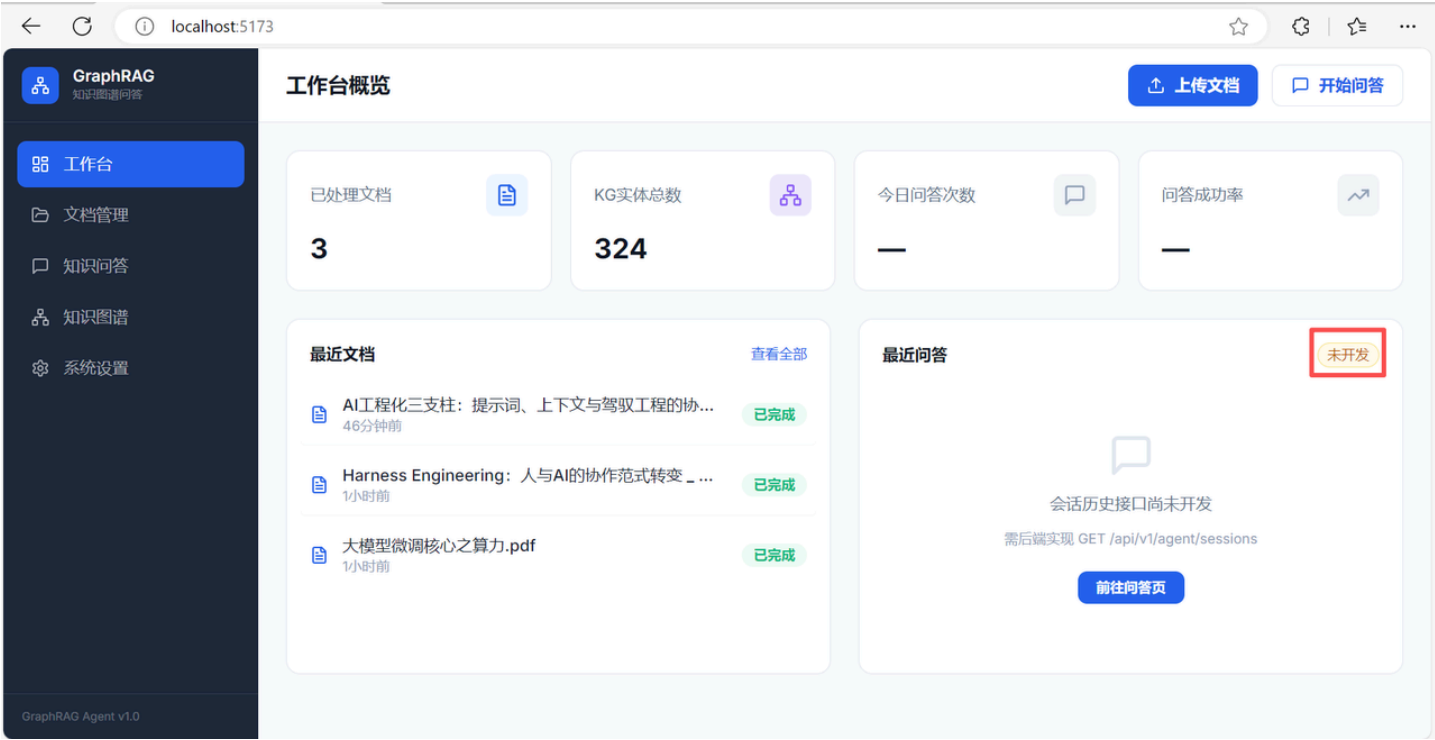
- 两个服务已在后台启动：
    - 后端：http://127.0.0.1:8000，日志在 /tmp/backend.log
    - 前端：http://localhost:5173，日志在 /tmp/frontend.log
- 如需确认是否正常运行，可以查看日志：cat /tmp/backend.log 或 cat /tmp/frontend.log。

\* Brewed for 40s

> █  
Model: Sonnet 4.6 (1M context) | Ctx: 37.8k | ⚡ no git | (no git)

10、效果验证

## 10.1 功能验证1：平台登录



我们开发完成的知识库平台已经可以正常访问和使用了。当然，受限于篇幅，还有部分扩展功能暂时没有完整实现，大家后续可以根据自身需求，直接让 Claude Code 继续完善和开发即可。另外，在实际体验和运行过程中，项目可能会出现一些报错提示，大家不用着急，只需要把完整的报错信息发给 Claude Code，它就能快速帮你定位并解决问题。

## 10.2 功能验证2：PDF文件解析

支持上传 PDF 文档并对其内容进行解析，该解析功能可正常生效。



## 10.3 功能验证3：知识图谱

支持查看解析完成后的知识图谱，功能及展示效果均正常。



如果觉得页面布局略显紧凑、排版拥挤，也可以继续与 Claude Code 交互，让它帮你优化界面样式、调整布局结构，让整体展示更清爽美观。

10.4 功能验证4：知识问答

问答功能测试结果正常。



整体实现出来的效果还是相当不错的，完全达到了我们预期的演示标准！

大家继续探索即可。

## 11、总结

本章节主要为大家讲解了如何基于 Claude Code 编写企业级项目。我们从两个核心角度，为大家分享了 Claude Code 的开发相关内容：一方面，在开发过程中，向大家传授了诸多 Claude Code 的实用使用技巧；另一方面，通过实际演示，完整展示了基于 Claude Code 开发企业级项目的全流程。

整个项目开发流程循序渐进，我们先进行项目准备、架构设计与架构学习，随后开展详细的技术调研。调研完成后，我们进行了多次 MVP 测试，这些测试至关重要——正是通过这些测试，我们建立了规范的开发标准。基于这套开发规范，我们进一步生成了产品 PRD 文档。

有了 PRD 文档作为基础，我们着手进行前端开发，在前端代码开发过程中，引入了 MCP 工具，助力打造出视觉效果优良的前端界面。紧接着，我们基于 PRD 文档开发后端代码，后端开发完成后，开展了前后端联调工作，并在联调过程中，向大家分享了诸多实用的实操经验。

所有流程完成后，我们进行了最终的效果验证，验证结果显示，项目最终呈现的效果与初始设定的项目目标完全一致。